```
// BootRunner.cs
using System.Collections;
using Sy.Core;
using UnityEngine;
using UnityEngine.SceneManagement;
using Logger = Sy.Core.Logger;
namespace Sy.G02.Boot
{
    public static class SceneIds
    {
        public const string Boot = "S00_Boot";
        public const string Main = "S02_Main";
    }
    /// &lt;summary&gt;
    /// S00_Boot scene host: register mock services -&gt; run seven steps -&gt; switch to main scene.
    /// On failure it stays on S00_Boot and logs the error (no dead-end while loops).
    /// &lt;/summary&gt;
    public sealed class BootRunner : MonoBehaviour
    {
        IEnumerator Start()
        {
            BootServices.RegisterAll();
            var sequence = new BootSequence(SwitchToMain);
            if (!sequence.RunAll())
            {
                Logger.Error("BOOT", "boot sequence failed - stay on S00_Boot");
                yield break;
            }
            yield return null;
        }
        bool SwitchToMain()
        {
            StartCoroutine(LoadMainRoutine());
            return true;
        }
        IEnumerator LoadMainRoutine()
        {
            var op = SceneManager.LoadSceneAsync(SceneIds.Main, LoadSceneMode.Single);
            while (op != null &amp;&amp; !op.isDone) yield return null;
        }
    }
}
// BootSequence.cs
using System;
using System.Collections.Generic;
using Sy.Core;
using Sy.Platform;
namespace Sy.G02.Boot
{
    /// &lt;summary&gt;
```

```
/// Seven-step boot sequence: Logger -&gt; Time -&gt; Config -&gt; Save -&gt; Locale -&gt; Ads.Preload -&gt; SwitchM
/// each step logs "[BOOT] n/7 name ok|FAIL". A step exception never blocks later steps.
/// &lt;/summary&gt;
public sealed class BootSequence
{
    public const int TotalSteps = 7;
    readonly Func&lt;bool&gt; _switchScene;
    readonly List&lt;string&gt; _lines = new();
    public BootSequence(Func&lt;bool&gt; switchScene)
    {
        _switchScene = switchScene;
    }
    public IReadOnlyList&lt;string&gt; Lines =&gt; _lines;
    public bool RunAll()
    {
        bool ok = true;
        ok &amp;= Run(1, "Logger", LoggerStep);
        ok &amp;= Run(2, "Time", TimeStep);
        ok &amp;= Run(3, "Config", ConfigStep);
        ok &amp;= Run(4, "Save", SaveStep);
        ok &amp;= Run(5, "ILocale", LocaleStep);
        ok &amp;= Run(6, "Ads.Preload", AdsStep);
        ok &amp;= Run(7, "SwitchMain", () =&gt; _switchScene != null &amp;&amp; _switchScene());
        return ok;
    }
    bool Run(int n, string name, Func&lt;bool&gt; step)
    {
        bool result;
        try
        {
            result = step();
        }
        catch (Exception e)
        {
            Logger.Error("BOOT", $"step {n} {name} exception: {e.Message}");
            result = false;
        }
        string line = $"[BOOT] {n}/{TotalSteps} {name} {(result ? "ok" : "FAIL")}";
        _lines.Add(line);
        Logger.Info("BOOT", line);
        return result;
    }
    static bool LoggerStep() =&gt; true; // static facade self-initializes on first call
    static bool TimeStep()
    {
        var time = ServiceLocator.Get&lt;ITimeService&gt;();
        return time.ServerNowMs() &gt; 0 &amp;&amp; !time.TimeCheated();
    }
    static bool ConfigStep()
    {
```

```
            var config = ServiceLocator.Get&lt;IConfigService&gt;();
            var update = config.CheckUpdate("mock://manifest");
            return update.NewVersion != null; // mock = no hot update
        }
        static bool SaveStep()
        {
            var save = ServiceLocator.Get&lt;ISaveService&gt;();
            save.Save("boot_probe", "{}");
            return !string.IsNullOrEmpty(save.Load("boot_probe"));
        }
        static bool LocaleStep()
        {
            var locale = ServiceLocator.Get&lt;ILocaleService&gt;();
            return !string.IsNullOrEmpty(locale.GetFontSet().NormalFont);
        }
        static bool AdsStep()
        {
            ServiceLocator.Get&lt;IAdsService&gt;().Preload("boot_warmup");
            return true;
        }
    }
}
// BootServices.cs
using System;
using System.Collections.Generic;
using Sy.Core;
using Sy.Platform;
using Sy.Telemetry;
using UnityEngine;
using Logger = Sy.Core.Logger;
namespace Sy.G02.Boot
{
    /// &lt;summary&gt;
    /// Dev-time mock infra services (SY_MOCK scope) plus the platform suite wiring
    /// (S2-T002). The four Sy.Core infra mocks (time/config/save/locale) stay local;
    /// every platform-adjacent service (ads/platform/storage) comes from the
    /// PlatformBootstrap suite so the editor path and a real WeChat build differ
    /// only by define, never by wiring. All seven boot steps depend on this set.
    /// &lt;/summary&gt;
    public static class BootServices
    {
        public static void RegisterAll()
        {
            if (ServiceLocator.TryGet&lt;ITimeService&gt;(out _)) return; // idempotent: double register would throw
            ServiceLocator.Register&lt;ITimeService&gt;(new MockTimeService());
            ServiceLocator.Register&lt;IConfigService&gt;(new MockConfigService());
            ServiceLocator.Register&lt;ISaveService&gt;(new MockSaveService());
            ServiceLocator.Register&lt;ILocaleService&gt;(new MockLocaleService());
            // S2-T002 platform suite: editor/SY_MOCK resolves the mock suite, real
            // WeChat builds resolve the Wx suite. Boot warm-up and the future offer
```

```
            // faces consume from the SAME policy instance, so pre-check
            // (HasCategoryBudget) and consumption stay consistent (P02 T006-B2
            // pattern). Category caps come from AdCapPolicy defaults: g02 rescue 4 /
            // double 5 on top of the 12/day global.
            var suite = PlatformBootstrap.Create(new AdCapPolicy());
            ServiceLocator.Register&lt;IAdsService&gt;(suite.Ads);
            ServiceLocator.Register&lt;IPlatformService&gt;(suite.Platform);
            ServiceLocator.Register&lt;IStorage&gt;(suite.Storage);
#if SY_WECHAT &amp;&amp; !UNITY_EDITOR
            // Wx custom analytics joins the telemetry pipeline via G02TelemetryWire.
            // Zero-server red line 12: the TCB batch sink class stays available in
            // the shared package but is NEVER registered here.
            ServiceLocator.Register&lt;IAnalyticsReporter&gt;(new G02WxAnalyticsReporter());
#endif
            Logger.Info("BOOT", "services registered (suite=" + PlatformBootstrap.SuiteIdFor(suite.Channel)
                + ", g02 ad categories: rescue 4 / double 5, global 12/day)");
        }
    }
    public sealed class MockTimeService : ITimeService
    {
        public TimeSource Source =&gt; TimeSource.Mock;
        public long ClientNowMs() =&gt; DateTimeOffset.UtcNow.ToUnixTimeMilliseconds();
        /// &lt;summary&gt;SY_MOCK: device time doubles as server time until the real chain lands.&lt;/summary&gt;
        public long ServerNowMs() =&gt; ClientNowMs();
        public bool TimeCheated() =&gt; Math.Abs(ServerNowMs() - ClientNowMs()) &gt; 120_000;
    }
    public sealed class MockConfigService : IConfigService
    {
        public T GetTable&lt;T&gt;(string tableId) where T : class
            =&gt; throw new KeyNotFoundException("mock config table not loaded: " + tableId);
        public bool TryGetTable&lt;T&gt;(string tableId, out T table) where T : class
        {
            table = null;
            return false;
        }
        public ConfigUpdateResult CheckUpdate(string manifestUrl)
            =&gt; new ConfigUpdateResult(false, "mock-0.0.1");
        public void SwapPolicy(ConfigSwapPolicy policy) { }
        public string PinConfigVersion(string scope) =&gt; "mock-0.0.1";
    }
    public sealed class MockSaveService : ISaveService
    {
        const string Prefix = "sy_save_";
        public void Save(string slot, string payloadJson) =&gt; PlayerPrefs.SetString(Prefix + slot, payloadJson);
        public string Load(string slot) =&gt; PlayerPrefs.GetString(Prefix + slot, string.Empty);
        /// &lt;summary&gt;No cloud: local storage is the authority (zero-server doctrine).&lt;/summary&gt;
        public SyncResult CloudSync() =&gt; new SyncResult(false, null);
    }
    public sealed class MockLocaleService : ILocaleService
    {
```

```
        public void SetLang(string lang) { }
        /// &lt;summary&gt;Missing key falls back to the key itself with a warning.&lt;/summary&gt;
        public string GetText(string key, params string[] args)
        {
            Logger.Warn("LOCALE", "missing i18n key (fallback): " + key);
            return key;
        }
        public FontSet GetFontSet()
            =&gt; new FontSet("G02Main", "G02Main", "G02Main");
    }
}
// GameCatalog.cs
using System;
using System.Collections.Generic;
using UnityEngine;
using Logger = Sy.Core.Logger;
namespace Sy.G02
{
    [Serializable]
    public sealed class EraData
    {
        public int id;
        public string years;
        public int unlockStars;
        public int repairFee;
        public float bgR;
        public float bgG;
        public float bgB;
        public string nameKey;
    }
    [Serializable]
    public sealed class PartData
    {
        public string id;
        public string icon;
        public int[] costs;
        public string nameKey;
    }
    [Serializable]
    public sealed class PhoneData
    {
        public string id;
        public int era;
        public string sprite;
        public string nameKey;
        public string triviaKey;
    }
    [Serializable]
    public sealed class FaultData
    {
```

```
    public string id;
    public string partId;
    public int minEra;
    public int maxEra;
    public string nameKey;
    public string[] symptomKeys;
    public string[] steps;
}
[Serializable]
public sealed class CustomerData
{
    public string[] nameKeys;
    public string[] happyQuipKeys;
    public string[] okQuipKeys;
    public string[] openLineKeys;
}
[Serializable]
public sealed class StepNameData
{
    public string screws;
    public string replace;
    public string clean;
    public string solder;
    public string assemble;
}
[Serializable]
public sealed class EconomyData
{
    public int twoFaultChance;
    public int penaltyPerMistakes;
    public int penaltyPct;
    public int minRewardPct;
    public int perfectStars;
    public int normalStars;
    public float doubleFaultFeeMult;
}
/// &lt;summary&gt;GDD s5 fresh-wallet stock seed: E1 starter parts (battery x2, antenna, keypad).&lt;/summary&gt;
[Serializable]
public sealed class StockSeedData
{
    public string id;
    public int count;
}
[Serializable]
public sealed class CatalogFile
{
    public EraData[] eras;
    public PartData[] parts;
    public PhoneData[] phones;
    public FaultData[] faults;
```

```
    public CustomerData customers;
    public StepNameData stepNames;
    public EconomyData economy;
    public StockSeedData[] initialStock;
}
/// &lt;summary&gt;
/// Static game catalog loaded from Resources/cfg_g02_catalog.json.
/// All display text lives behind i18n keys (red line 4: zero hardcoded Chinese in source).
/// &lt;/summary&gt;
public sealed class GameCatalog
{
    public EraData[] Eras { get; }
    public PartData[] Parts { get; }
    public PhoneData[] Phones { get; }
    public FaultData[] Faults { get; }
    public CustomerData Customers { get; }
    public StepNameData StepNames { get; }
    public EconomyData Economy { get; }
    public StockSeedData[] InitialStock { get; }
    GameCatalog(CatalogFile file)
    {
        Eras = file.eras;
        Parts = file.parts;
        Phones = file.phones;
        Faults = file.faults;
        Customers = file.customers;
        StepNames = file.stepNames;
        Economy = file.economy;
        InitialStock = file.initialStock;
    }
    public static GameCatalog Load()
    {
        var asset = Resources.Load&lt;TextAsset&gt;("cfg_g02_catalog");
        if (asset == null)
        {
            Logger.Warn("G02.Catalog", "cfg_g02_catalog.json not found under Resources");
            return null;
        }
        var file = JsonUtility.FromJson&lt;CatalogFile&gt;(asset.text);
        if (file == null || file.eras == null || file.eras.Length == 0)
        {
            Logger.Warn("G02.Catalog", "cfg_g02_catalog.json failed to parse");
            return null;
        }
        return new GameCatalog(file);
    }
    public EraData Era(int id)
    {
        return (id &gt;= 0 &amp;&amp; id &lt; Eras.Length) ? Eras[id] : null;
    }
```

```
public PartData Part(string id)
{
    foreach (var p in Parts)
    {
        if (p.id == id) return p;
    }
    return null;
}
public PhoneData Phone(string id)
{
    foreach (var p in Phones)
    {
        if (p.id == id) return p;
    }
    return null;
}
public FaultData Fault(string id)
{
    foreach (var f in Faults)
    {
        if (f.id == id) return f;
    }
    return null;
}
public bool FaultFitsEra(FaultData fault, int era)
{
    return era &gt;= fault.minEra &amp;&amp; era &lt;= fault.maxEra;
}
public List&lt;FaultData&gt; FaultsForEra(int era)
{
    var list = new List&lt;FaultData&gt;();
    foreach (var f in Faults)
    {
        if (FaultFitsEra(f, era)) list.Add(f);
    }
    return list;
}
public string[] PhoneIdsForEra(int era)
{
    var list = new List&lt;string&gt;();
    foreach (var p in Phones)
    {
        if (p.era == era) list.Add(p.id);
    }
    return list.ToArray();
}
/// &lt;summary&gt;Number of eras unlocked for a given reputation star total.&lt;/summary&gt;
public int UnlockedEraCount(int stars)
{
    int n = 1;
```

```
            for (int i = 1; i &lt; Eras.Length; i++)
            {
                if (stars &gt;= Eras[i].unlockStars) n = i + 1;
            }
            return n;
        }
        public string StepNameKey(string step)
        {
            switch (step)
            {
                case "screws": return StepNames.screws;
                case "replace": return StepNames.replace;
                case "clean": return StepNames.clean;
                case "solder": return StepNames.solder;
                case "assemble": return StepNames.assemble;
                default: return step;
            }
        }
    }
}
// G02AdPoints.cs
namespace Sy.G02.Boot
{
    /// &lt;summary&gt;
    /// G02 rewarded point ids (S2-T002). Namespaced with the g02_ prefix so no G02
    /// call can ever draw P02/P06 category budget and vice versa. Category mapping
    /// lives in Sy.Platform.AdCategories.CategoryOfPoint: RescueRetry = the
    /// "almost-through" fail offer at settle (restore reward percent),
    /// SettleDouble = the settle pay doubling offer. Caps come from AdCapPolicy
    /// defaults: rescue 4 / double 5 per day on top of the 12/day global.
    /// &lt;/summary&gt;
    public static class G02AdPoints
    {
        public const string RescueRetry = "g02_rescue_retry";
        public const string SettleDouble = "g02_settle_double";
    }
}
// G02WxAnalyticsReporter.cs
#if SY_WECHAT
using System.Runtime.InteropServices;
using Sy.Telemetry;
namespace Sy.G02.Boot
{
    /// &lt;summary&gt;
    /// WeChat analytics reporter (S2-T002): bridges the telemetry WxAnalyticsSink onto
    /// the platform JS entry SyReportAnalytics declared in SyWxBridge.jslib. Compiled
    /// only for real WeChat builds (no UNITY_EDITOR); mock/editor builds compile this
    /// out entirely and the debug sink stays the only one. The duplicate extern
    /// declaration binds to the same jslib function the Sy.Platform Wx bridge uses.
    /// &lt;/summary&gt;
```

```
    public sealed class G02WxAnalyticsReporter : IAnalyticsReporter
    {
        [DllImport("__Internal", EntryPoint = "SyReportAnalytics")]
        static extern void ReportBridge(string evtId, string fieldsJson);
        public void Report(string evtId, string fieldsJson)
            =&gt; ReportBridge(evtId ?? "", fieldsJson ?? "{}");
    }
}
#endif
// Economy.cs
using UnityEngine;
namespace Sy.G02
{
    /// &lt;summary&gt;
    /// Repair economy math (GDD section 5 + prototype parity):
    /// reward pct = 100 - floor(mistakes / penaltyPerMistakes) * penaltyPct, floored at minRewardPct;
    /// fee = era.repairFee * (faults &gt;= 2 ? doubleFaultFeeMult : 1) * rewardPct / 100, rounded;
    /// stars = perfectStars when zero mistakes else normalStars.
    /// &lt;/summary&gt;
    public static class Economy
    {
        public static int RewardPct(int mistakes, GameCatalog catalog)
        {
            int q = 100 - (mistakes / catalog.Economy.penaltyPerMistakes) * catalog.Economy.penaltyPct;
            return Mathf.Max(catalog.Economy.minRewardPct, q);
        }
        public static int FeeFor(int eraId, int faultCount, int mistakes, GameCatalog catalog)
        {
            var era = catalog.Era(eraId);
            if (era == null) return 0;
            float fee = era.repairFee * (faultCount &gt;= 2 ? catalog.Economy.doubleFaultFeeMult : 1f);
            return Mathf.RoundToInt(fee * RewardPct(mistakes, catalog) / 100f);
        }
        public static int StarsFor(int mistakes, GameCatalog catalog)
        {
            return mistakes == 0 ? catalog.Economy.perfectStars : catalog.Economy.normalStars;
        }
    }
}
// G02ChapterGate.cs
using Sy.Core;
namespace Sy.G02
{
    /// &lt;summary&gt;
    /// Chapter unlock moment state machine (task S2-T006 slice A, GDD s2/s5):
    /// converts a reputation star total into era unlocks against the previously
    /// persisted era count (S2-T001 save linkage), publishing one EraUnlockedEvent
    /// per newly crossed threshold in ascending era order. Deterministic and
    /// headless-testable: the T006-B ceremony view subscribes to the event and the
    /// caller persists the returned era count through G02SaveSystem.Persist.
```

```
    /// Era ids are 0-based (era 0 = E1, always unlocked).
    /// &lt;/summary&gt;
    public static class G02ChapterGate
    {
        /// &lt;summary&gt;
        /// Era ids that the star total unlocks beyond the previous count, ascending.
        /// Era 1 is always unlocked, so the previous count is clamped to at least 1.
        /// &lt;/summary&gt;
        public static int[] NewlyUnlocked(int previousEraCount, int starsTotal, GameCatalog catalog)
        {
            if (catalog == null) return new int[0];
            int previous = previousEraCount &lt; 1 ? 1 : previousEraCount;
            int current = catalog.UnlockedEraCount(starsTotal &lt; 0 ? 0 : starsTotal);
            if (current &lt;= previous) return new int[0];
            var ids = new int[current - previous];
            for (int i = previous; i &lt; current; i++) ids[i - previous] = i;
            return ids;
        }
        /// &lt;summary&gt;
        /// Apply the unlock check and return the era count to persist. Publishes one
        /// EraUnlockedEvent per newly unlocked era in ascending order; a null bus is
        /// tolerated (count-only pass, no throw) so callers without a bus stay safe.
        /// &lt;/summary&gt;
        public static int ApplyUnlocks(SyEventBus bus, int previousEraCount, int starsTotal, GameCatalog catalog)
        {
            if (catalog == null) return previousEraCount &lt; 1 ? 1 : previousEraCount;
            int previous = previousEraCount &lt; 1 ? 1 : previousEraCount;
            int current = catalog.UnlockedEraCount(starsTotal &lt; 0 ? 0 : starsTotal);
            if (bus != null)
            {
                for (int i = previous; i &lt; current; i++)
                {
                    var era = catalog.Era(i);
                    bus.Publish(new EraUnlockedEvent
                    {
                        eraId = i,
                        years = era != null ? era.years : "",
                        unlockStars = era != null ? era.unlockStars : 0,
                        starsTotal = starsTotal &lt; 0 ? 0 : starsTotal,
                    });
                }
            }
            return current &gt; previous ? current : previous;
        }
    }
}
// G02Sfx.cs
using System;
using System.Collections.Generic;
using Sy.Core;
```

```
using UnityEngine;
using Logger = Sy.Core.Logger;
namespace Sy.G02
{
    /// &lt;summary&gt;
    /// Procedural sfx + signature leitmotif for G02 (task S2-T003, GDD s7 zero-audio-file
    /// doctrine: AudioClip.Create runtime synthesis, no shipped audio assets - quota-mode
    /// legal). All synthesis is pure math over a float[] buffer: deterministic, EditMode
    /// testable, no Random, no files.
    ///
    /// G02 signature leitmotif (earworm red line: hummable within 3s, unique across the
    /// portfolio): "wind-up triple rise + bell" - three rising plucked notes E5(659.3Hz)
    /// -&gt; G5(784.0Hz) -&gt; B5(987.8Hz), each 0.18s, a 0.12s breath, then a bright E6
    /// (1318.5Hz) bell decay. It mirrors the shop fantasy: winding up a nostalgic phone,
    /// and it dings when the repair lands. Same hook family feeds future BGM slices.
    /// &lt;/summary&gt;
    public static class G02Sfx
    {
        public const int SampleRate = 22050;
        public const string ScrewTap = "screw";       // screw-tighten ratchet click (per tap)
        public const string PartSnap = "snap";        // part seating thunk (correct drop)
        public const string StepWhir = "whir";        // spin-out swoosh (perfect step done)
        public const string SettleBell = "settle";    // settle-done chime (reward screen)
        public const string MissTone = "miss";        // weak mistake thud
        public const string Leitmotif = "leitmotif";  // signature hook (menu branding)
        static readonly Dictionary&lt;string, AudioClip&gt; ClipCache = new();
        static AudioSource _host;
        static readonly Dictionary&lt;string, int&gt; _playAttempts = new();
        /// &lt;summary&gt;Audio-intent census counter (U153-D05a): Play call count per clip id.
        /// Attempt semantics - incremented before playback, so headless runs with no
        /// live audio device still record the feedback-intent channel honestly.&lt;/summary&gt;
        public static int PlayAttempts(string clipId)
        {
            return _playAttempts.TryGetValue(clipId, out int n) ? n : 0;
        }
        public static int PlayAttemptsTotal()
        {
            int total = 0;
            foreach (var kv in _playAttempts) total += kv.Value;
            return total;
        }
        public static void ResetPlayAttempts()
        {
            _playAttempts.Clear();
        }
        /// &lt;summary&gt;Deterministic sample synthesis for one clip id (pure, no Unity objects).&lt;/summary&gt;
        public static float[] Synth(string clipId)
        {
            switch (clipId)
            {
```

```
        case ScrewTap: return ScrewSamples();
        case PartSnap: return SnapSamples();
        case StepWhir: return WhirSamples();
        case SettleBell: return SettleSamples();
        case MissTone: return MissSamples();
        case Leitmotif: return LeitmotifSamples();
        default: return new float[0];
    }
}
/// &lt;summary&gt;Cached AudioClip wrapper for playback (main thread only).&lt;/summary&gt;
public static AudioClip Clip(string clipId)
{
    if (ClipCache.TryGetValue(clipId, out var cached) &amp;&amp; cached != null) return cached;
    var data = Synth(clipId);
    if (data.Length == 0) return null;
    var clip = AudioClip.Create("g02_sfx_" + clipId, data.Length, 1, SampleRate, false);
    clip.SetData(data, 0);
    ClipCache[clipId] = clip;
    return clip;
}
/// &lt;summary&gt;Fire-and-forget playback on a hidden host; never throws in headless runs.
/// Edit-mode/headless runs carry no live audio path (DontDestroyOnLoad is a
/// play-mode concept), so the Unity host is only built while playing - the
/// attempt counter above still records the intent channel (U153-D05a census).
/// Device builds and editor play mode are unchanged.&lt;/summary&gt;
public static void Play(string clipId)
{
    _playAttempts[clipId] = PlayAttempts(clipId) + 1;
    if (!Application.isPlaying) return;
    try
    {
        if (_host == null)
        {
            var go = new GameObject("G02SfxHost");
            UnityEngine.Object.DontDestroyOnLoad(go);
            _host = go.AddComponent&lt;AudioSource&gt;();
        }
        var clip = Clip(clipId);
        if (clip != null) _host.PlayOneShot(clip);
    }
    catch (Exception e)
    {
        Logger.Warn("G02.Sfx", "play skipped (" + clipId + "): " + e.Message);
    }
}
// ---- synthesis recipes ----------------------------------------------------
/// &lt;summary&gt;Screw-tighten ratchet: 8ms filtered-noise burst + 2.1kHz ping, fast decay.&lt;/summary&gt;
static float[] ScrewSamples()
{
    int n = (int)(SampleRate * 0.07);
```

```
        var buf = new float[n];
        for (int i = 0; i &lt; n; i++)
        {
            double t = (double)i / SampleRate;
            double env = Math.Exp(-t / 0.012);
            double noise = Frac(Math.Sin(i * 12.9898) * 43758.5453) * 2.0 - 1.0;
            double ping = Math.Sin(2.0 * Math.PI * 2100.0 * t) * 0.5;
            buf[i] = (float)((noise * 0.7 + ping) * env);
        }
        return Normalize(buf, 0.80);
    }
    /// &lt;summary&gt;Part snap: 220-&gt;150Hz descending thunk with a click transient.&lt;/summary&gt;
    static float[] SnapSamples()
    {
        int n = (int)(SampleRate * 0.12);
        var buf = new float[n];
        for (int i = 0; i &lt; n; i++)
        {
            double t = (double)i / SampleRate;
            double env = Math.Exp(-t / 0.030);
            double freq = 220.0 - 70.0 * (t / 0.12);
            double body = Math.Sin(2.0 * Math.PI * freq * t);
            double click = i &lt; SampleRate / 400 ? Frac(Math.Sin(i * 7.77) * 123.45) * 2.0 - 1.0 : 0.0;
            buf[i] = (float)((body * 0.85 + click * 0.4) * env);
        }
        return Normalize(buf, 0.80);
    }
    /// &lt;summary&gt;Spin-out whir: 900-&gt;300Hz sweep with vibrato, fading tail.&lt;/summary&gt;
    static float[] WhirSamples()
    {
        int n = (int)(SampleRate * 0.20);
        var buf = new float[n];
        for (int i = 0; i &lt; n; i++)
        {
            double t = (double)i / SampleRate;
            double env = Math.Exp(-t / 0.08);
            double freq = 900.0 - 600.0 * (t / 0.20);
            double vibrato = 1.0 + 0.02 * Math.Sin(2.0 * Math.PI * 38.0 * t);
            buf[i] = (float)(Math.Sin(2.0 * Math.PI * freq * vibrato * t) * env);
        }
        return Normalize(buf, 0.60);
    }
    /// &lt;summary&gt;Settle bell: E6 + B5 fifth partials with a faster octave sparkle.&lt;/summary&gt;
    static float[] SettleSamples()
    {
        int n = (int)(SampleRate * 0.45);
        var buf = new float[n];
        for (int i = 0; i &lt; n; i++)
        {
            double t = (double)i / SampleRate;
```

```
            double e6 = Math.Sin(2.0 * Math.PI * 1318.5 * t) * Math.Exp(-t / 0.16);
            double b5 = Math.Sin(2.0 * Math.PI * 987.8 * t) * Math.Exp(-t / 0.11) * 0.6;
            double sparkle = Math.Sin(2.0 * Math.PI * 2637.0 * t) * Math.Exp(-t / 0.05) * 0.25;
            buf[i] = (float)(e6 + b5 + sparkle);
        }
        return Normalize(buf, 0.75);
    }
    /// &lt;summary&gt;Mistake weak tone: soft 220Hz thud, deliberately quieter than the core trio.&lt;/summary&gt;
    static float[] MissSamples()
    {
        int n = (int)(SampleRate * 0.09);
        var buf = new float[n];
        for (int i = 0; i &lt; n; i++)
        {
            double t = (double)i / SampleRate;
            double env = Math.Exp(-t / 0.028);
            buf[i] = (float)(Math.Sin(2.0 * Math.PI * 220.0 * t) * env);
        }
        return Normalize(buf, 0.45);
    }
    /// &lt;summary&gt;
    /// Signature leitmotif "wind-up triple rise + bell": E5 -&gt; G5 -&gt; B5 plucked rise,
    /// breath, E6 bell. ~1.6s, inside the 3s hummable red line.
    /// &lt;/summary&gt;
    static float[] LeitmotifSamples()
    {
        const double noteLen = 0.18;
        const double breath = 0.12;
        const double bellLen = 0.90;
        int total = (int)(SampleRate * (noteLen * 3.0 + breath + bellLen));
        var buf = new float[total];
        double[] steps = { 659.3, 784.0, 987.8 };
        for (int s = 0; s &lt; 3; s++)
        {
            int start = (int)(SampleRate * (noteLen * s));
            int len = (int)(SampleRate * noteLen);
            double freq = steps[s];
            for (int i = 0; i &lt; len &amp;&amp; start + i &lt; total; i++)
            {
                double t = (double)i / SampleRate;
                double env = Math.Exp(-t / 0.10);
                double vibrato = 1.0 + 0.004 * Math.Sin(2.0 * Math.PI * 5.5 * t);
                buf[start + i] = (float)(Math.Sin(2.0 * Math.PI * freq * vibrato * t) * env);
            }
        }
        int bellStart = (int)(SampleRate * (noteLen * 3.0 + breath));
        for (int i = 0; bellStart + i &lt; total; i++)
        {
            double t = (double)i / SampleRate;
            double env = Math.Exp(-t / 0.30);
```

```
                double bell = Math.Sin(2.0 * Math.PI * 1318.5 * t)
                            + Math.Sin(2.0 * Math.PI * 2637.0 * t) * Math.Exp(-t / 0.10) * 0.4;
                buf[bellStart + i] = (float)(bell * env);
            }
            return Normalize(buf, 0.80);
        }
        static double Frac(double x)
        {
            return x - Math.Floor(x);
        }
        /// &lt;summary&gt;Peak normalization to a fixed target - keeps the loudness family tight.&lt;/summary&gt;
        static float[] Normalize(float[] buf, double targetPeak)
        {
            double peak = 0.0;
            for (int i = 0; i &lt; buf.Length; i++)
            {
                double a = Math.Abs(buf[i]);
                if (a &gt; peak) peak = a;
            }
            if (peak &lt; 1e-6) return buf;
            double scale = targetPeak / peak;
            for (int i = 0; i &lt; buf.Length; i++) buf[i] = (float)(buf[i] * scale);
            return buf;
        }
    }
}
// G02Text.cs
using System;
using System.Collections.Generic;
using UnityEngine;
using Logger = Sy.Core.Logger;
namespace Sy.G02
{
    [Serializable]
    public sealed class I18nEntry
    {
        public string key;
        public string text;
    }
    [Serializable]
    public sealed class I18nFile
    {
        public I18nEntry[] entries;
    }
    /// &lt;summary&gt;
    /// Project i18n table (red line 4): all player-facing Chinese text is externalized
    /// into Resources/cfg_g02_i18n.json. Missing key falls back to the key itself so the
    /// game never crashes on text lookup.
    /// &lt;/summary&gt;
    public static class G02Text
```

```
    {
        static Dictionary&lt;string, string&gt; _map;
        public static void Load()
        {
            _map = new Dictionary&lt;string, string&gt;();
            var asset = Resources.Load&lt;TextAsset&gt;("cfg_g02_i18n");
            if (asset == null)
            {
                Logger.Warn("G02.I18n", "cfg_g02_i18n.json not found under Resources");
                return;
            }
            var file = JsonUtility.FromJson&lt;I18nFile&gt;(asset.text);
            if (file == null || file.entries == null)
            {
                Logger.Warn("G02.I18n", "cfg_g02_i18n.json failed to parse");
                return;
            }
            foreach (var e in file.entries)
            {
                if (!string.IsNullOrEmpty(e.key)) _map[e.key] = e.text;
            }
        }
        static void EnsureLoaded()
        {
            if (_map == null) Load();
        }
        public static string Get(string key)
        {
            EnsureLoaded();
            if (_map.TryGetValue(key, out var text) &amp;&amp; !string.IsNullOrEmpty(text)) return text;
            Logger.Warn("G02.I18n", "missing i18n key (fallback to key): " + key);
            return key;
        }
        public static bool Has(string key)
        {
            EnsureLoaded();
            return _map.TryGetValue(key, out var text) &amp;&amp; !string.IsNullOrEmpty(text);
        }
    }
}
// RepairOrderGenerator.cs
using System;
using System.Collections.Generic;
namespace Sy.G02
{
    /// &lt;summary&gt;One customer repair order. Deterministic for a given seed.&lt;/summary&gt;
    [Serializable]
    public sealed class RepairOrder
    {
        public int eraId;
```

```
        public string phoneId;
        public string[] faultIds;
        public string customerNameKey;
        public string openLineKey;
    }
    /// &lt;summary&gt;
    /// Seeded repair-order generator (prototype Game.cs parity):
    /// era uniform over unlocked eras, phone uniform in era, 1-2 unique faults
    /// filtered by era range, random customer and opening line.
    /// &lt;/summary&gt;
    public static class RepairOrderGenerator
    {
        public static RepairOrder Generate(int seed, GameCatalog catalog, int unlockedEraCount)
        {
            var rng = new System.Random(seed);
            int eraCap = Math.Max(1, unlockedEraCount);
            int era = rng.Next(eraCap);
            var phoneIds = catalog.PhoneIdsForEra(era);
            string phoneId = phoneIds[rng.Next(phoneIds.Length)];
            int faultTarget = rng.Next(100) &lt; catalog.Economy.twoFaultChance ? 2 : 1;
            var pool = catalog.FaultsForEra(era);
            var picked = new List&lt;string&gt;();
            while (picked.Count &lt; faultTarget &amp;&amp; picked.Count &lt; pool.Count)
            {
                var candidate = pool[rng.Next(pool.Count)];
                if (!picked.Contains(candidate.id)) picked.Add(candidate.id);
            }
            string nameKey = catalog.Customers.nameKeys[rng.Next(catalog.Customers.nameKeys.Length)];
            string openKey = catalog.Customers.openLineKeys[rng.Next(catalog.Customers.openLineKeys.Length)];
            return new RepairOrder
            {
                eraId = era,
                phoneId = phoneId,
                faultIds = picked.ToArray(),
                customerNameKey = nameKey,
                openLineKey = openKey,
            };
        }
    }
}
// G02Events.cs
namespace Sy.G02
{
    /// &lt;summary&gt;Telemetry event ids for G02; cfg_g02_events.json mirrors this set.&lt;/summary&gt;
    public static class G02EventIds
    {
        public const string NewbieStep = "evt_newbie_step";
        public const string RepairFunnel = "evt_repair_funnel";
    }
    /// &lt;summary&gt;Funnel phase labels used as evt_repair_funnel.phase values.&lt;/summary&gt;
```

```
public static class G02FunnelPhases
{
    public const string SessionStart = "session_start";
    public const string Diagnosis = "diagnosis";
    public const string RepairStart = "repair_start";
    public const string Step = "step";
    public const string Settle = "settle";
    public const string SessionClose = "session_close";
}
/// &lt;summary&gt;Menu -&gt; Desk: a customer arrived, order rolled, parts charged.&lt;/summary&gt;
public sealed class SessionStartedEvent
{
    public int seed;
    public int eraId;
    public string phoneId;
    public int faultCount;
    public int totalSteps;
    public bool partCharged;
    public int partChargeCost;
}
/// &lt;summary&gt;Desk -&gt; Diagnosis.&lt;/summary&gt;
public sealed class DiagnosisConfirmedEvent { }
/// &lt;summary&gt;Diagnosis -&gt; Repair.&lt;/summary&gt;
public sealed class RepairStartedEvent { }
/// &lt;summary&gt;One repair work step finished (perfect or counted as a mistake).&lt;/summary&gt;
public sealed class StepCompletedEvent
{
    public int stepIndex;
    public bool perfect;
    public int mistakes;
}
/// &lt;summary&gt;Repair -&gt; Settle: economy result snapshot.&lt;/summary&gt;
public sealed class SessionSettledEvent
{
    public string phoneId;
    public int pay;
    public int starsGained;
    public bool perfect;
    public int mistakes;
}
/// &lt;summary&gt;Settle -&gt; Menu: session closed.&lt;/summary&gt;
public sealed class SessionClosedEvent { }
/// &lt;summary&gt;
/// Chapter unlock moment (task S2-T006): the reputation star total crossed an era
/// unlock threshold. Published ascending per newly unlocked era; the T006-B ceremony
/// view (new-era lightbox opening ritual) subscribes to this event.
/// &lt;/summary&gt;
public sealed class EraUnlockedEvent
{
    public int eraId;
```

```
        public string years;
        public int unlockStars;
        public int starsTotal;
    }
}
// G02MiniGames.cs
using System;
using UnityEngine;
namespace Sy.G02
{
    /// &lt;summary&gt;
    /// Shared contract for the five repair minigame rule cores (task S1-T003 slice B).
    /// Pure logic: the view feeds pointer events in stage-local coordinates, the core
    /// owns hit tests, mistake counting and completion. Zero UI references so EditMode
    /// tests cover every rule. Mistake accounting is per-step: any mistake inside one
    /// step makes that step imperfect (CompleteStep(false)), the economy divides by
    /// penaltyPerMistakes downstream (GDD section 5).
    /// &lt;/summary&gt;
    public abstract class MiniGameLogic
    {
        public bool Done { get; protected set; }
        public int Mistakes { get; protected set; }
        public bool Perfect { get { return Mistakes == 0; } }
        /// &lt;summary&gt;0..1 progress for HUD feedback.&lt;/summary&gt;
        public abstract float Progress { get; }
        /// &lt;summary&gt;Raised exactly once when the game completes.&lt;/summary&gt;
        public event Action Completed;
        protected void Complete()
        {
            if (Done) return;
            Done = true;
            if (Completed != null) Completed();
        }
    }
    /// &lt;summary&gt;
    /// Screws: tap each screw N times to spin it out. Ordered mode follows the GDD rule
    /// (tap in order, off-order taps are ignored, no mistake concept); unordered mode is
    /// the assemble variant (prototype ReassembleMinigame parity: any screw, one tap).
    /// &lt;/summary&gt;
    public sealed class ScrewsGame : MiniGameLogic
    {
        readonly Vector2[] _positions;
        readonly int[] _taps;
        readonly int _need;
        readonly bool _ordered;
        int _current;
        int _doneCount;
        public ScrewsGame(Vector2[] positions, int tapsPerScrew, bool ordered)
        {
            if (positions == null || positions.Length == 0)
```

```
            throw new ArgumentException("screws need at least one position", "positions");
        _positions = positions;
        _taps = new int[positions.Length];
        _need = Mathf.Max(1, tapsPerScrew);
        _ordered = ordered;
    }
    public int ScrewCount { get { return _positions.Length; } }
    public int DoneCount { get { return _doneCount; } }
    public int TapsNeeded { get { return _need; } }
    /// &lt;summary&gt;Index of the screw the player must tap next (ordered mode).&lt;/summary&gt;
    public int CurrentScrew { get { return Mathf.Min(_current, _positions.Length - 1); } }
    public Vector2 PositionOf(int index) { return _positions[index]; }
    public int TapsOn(int index) { return _taps[index]; }
    public override float Progress
    {
        get
        {
            float total = _positions.Length * _need;
            if (total &lt;= 0f) return 1f;
            float done = 0f;
            for (int i = 0; i &lt; _taps.Length; i++) done += Mathf.Min(_taps[i], _need);
            return done / total;
        }
    }
    /// &lt;summary&gt;Tap at a stage-local point; returns the hit screw index or -1 for a blank tap.&lt;/summary&gt
    public int Tap(Vector2 point, float radius)
    {
        if (Done) return -1;
        for (int i = 0; i &lt; _positions.Length; i++)
        {
            if (!Within(point, _positions[i], radius)) continue;
            if (_ordered &amp;&amp; i != CurrentScrew) return -1; // off-order tap: ignored
            if (_taps[i] &gt;= _need) return -1;             // already spun out
            _taps[i]++;
            if (_taps[i] &gt;= _need)
            {
                _doneCount++;
                if (_ordered) _current++;
                if (_doneCount &gt;= _positions.Length) Complete();
            }
            return i;
        }
        return -1;
    }
    static bool Within(Vector2 a, Vector2 b, float radius)
    {
        return Mathf.Abs(a.x - b.x) &lt; radius &amp;&amp; Mathf.Abs(a.y - b.y) &lt; radius;
    }
}
/// &lt;summary&gt;
```

```
/// Replace: drop one of three candidates into the slot. A wrong part snaps back and
/// counts one mistake (prototype parity: every wrong drop is one mistake). Releasing
/// away from the slot is a free miss with no penalty.
/// &lt;/summary&gt;
public sealed class ReplaceGame : MiniGameLogic
{
    readonly Vector2 _slot;
    readonly int _correct;
    readonly float _snap;
    public ReplaceGame(Vector2 slot, int correctIndex, float snapRadius)
    {
        _slot = slot;
        _correct = correctIndex;
        _snap = Mathf.Max(1f, snapRadius);
    }
    public int CorrectIndex { get { return _correct; } }
    public Vector2 Slot { get { return _slot; } }
    public override float Progress { get { return Done ? 1f : 0f; } }
    /// &lt;summary&gt;Release candidate at a stage-local point; true when the right part landed.&lt;/summary&gt;
    public bool Drop(int candidateIndex, Vector2 at)
    {
        if (Done) return false;
        if (candidateIndex &lt; 0) return false;
        if (Vector2.Distance(at, _slot) &gt; _snap) return false; // missed the slot
        if (candidateIndex == _correct)
        {
            Complete();
            return true;
        }
        Mistakes++;
        return false;
    }
}
/// &lt;summary&gt;
/// Clean: wipe dirt cells on a grid mapped from the stage-local board rect. Dropping
/// below the leftover threshold finishes the step (GDD: about 95% wiped, no mistakes).
/// &lt;/summary&gt;
public sealed class CleanGame : MiniGameLogic
{
    readonly bool[] _dirt;
    readonly Rect _board;
    readonly int _cols;
    readonly int _rows;
    readonly int _threshold;
    int _left;
    public CleanGame(bool[] dirt, int cols, int rows, Rect board, int leftoverThreshold)
    {
        if (dirt == null || dirt.Length != cols * rows || cols &lt;= 0 || rows &lt;= 0)
            throw new ArgumentException("dirt mask must match cols x rows", "dirt");
        _dirt = dirt;
```

```
            _cols = cols;
            _rows = rows;
            _board = board;
            _threshold = Mathf.Max(0, leftoverThreshold);
            _left = 0;
            for (int i = 0; i &lt; dirt.Length; i++) if (dirt[i]) _left++;
            InitialDirt = _left;
            if (_left &lt;= _threshold) Complete();
        }
        public int Cols { get { return _cols; } }
        public int Rows { get { return _rows; } }
        public int DirtLeft { get { return _left; } }
        public int InitialDirt { get; private set; }
        public bool IsDirty(int index) { return _dirt[index]; }
        public override float Progress
        {
            get { return InitialDirt &lt;= 0 ? 1f : 1f - (float)_left / InitialDirt; }
        }
        /// &lt;summary&gt;Wipe at a stage-local point with a round brush measured in cells.&lt;/summary&gt;
        public int Wipe(Vector2 point, int brushCells)
        {
            if (Done) return 0;
            if (!_board.Contains(point)) return 0;
            int cx = Mathf.FloorToInt((point.x - _board.x) / _board.width * _cols);
            int cy = Mathf.FloorToInt((point.y - _board.y) / _board.height * _rows);
            int cleared = 0;
            for (int y = cy - brushCells; y &lt;= cy + brushCells; y++)
            {
                for (int x = cx - brushCells; x &lt;= cx + brushCells; x++)
                {
                    if (x &lt; 0 || y &lt; 0 || x &gt;= _cols || y &gt;= _rows) continue;
                    if ((x - cx) * (x - cx) + (y - cy) * (y - cy) &gt; brushCells * brushCells) continue;
                    int i = y * _cols + x;
                    if (!_dirt[i]) continue;
                    _dirt[i] = false;
                    _left--;
                    cleared++;
                }
            }
            if (_left &lt;= _threshold) Complete();
            return cleared;
        }
    }
    /// &lt;summary&gt;
    /// Solder: press and drag through the point sequence in order. Releasing mid-sequence
    /// resets the sequence and counts one mistake; releasing before the first point or
    /// after the last one is free (prototype SolderMinigame parity).
    /// &lt;/summary&gt;
    public sealed class SolderGame : MiniGameLogic
    {
```

```
        readonly Vector2[] _points;
        readonly float _radius;
        int _index;
        bool _wasPressed;
        public SolderGame(Vector2[] points, float radius)
        {
            if (points == null || points.Length == 0)
                throw new ArgumentException("solder needs at least one point", "points");
            _points = points;
            _radius = Mathf.Max(1f, radius);
        }
        public int Index { get { return _index; } }
        public int PointCount { get { return _points.Length; } }
        public Vector2 PointOf(int index) { return _points[index]; }
        public override float Progress
        {
            get { return (float)_index / _points.Length; }
        }
        /// &lt;summary&gt;Feed pointer state each frame; true when a solder point advanced.&lt;/summary&gt;
        public bool Track(Vector2 point, bool pressed)
        {
            if (Done) return false;
            bool advanced = false;
            if (pressed)
            {
                var p = _points[Mathf.Min(_index, _points.Length - 1)];
                if (Mathf.Abs(point.x - p.x) &lt; _radius &amp;&amp; Mathf.Abs(point.y - p.y) &lt; _radius)
                {
                    _index++;
                    advanced = true;
                    if (_index &gt;= _points.Length)
                    {
                        Complete();
                        return true;
                    }
                }
            }
            else if (_wasPressed &amp;&amp; _index &gt; 0 &amp;&amp; _index &lt; _points.Length)
            {
                _index = 0;
                Mistakes++;
            }
            _wasPressed = pressed;
            return advanced;
        }
    }
}
// G02SaveSystem.cs
using System;
using System.Collections.Generic;
```

```
using Sy.Core;
namespace Sy.G02
{
    /// &lt;summary&gt;Outcome of a save-slot load (task S2-T001).&lt;/summary&gt;
    public enum G02LoadStatus
    {
        NewGame,         // slot empty: fresh wallet, nothing written yet
        Loaded,          // current-version payload applied as-is
        Migrated,        // older payload upgraded in place (defaults for missing fields)
        CorruptedReset   // unreadable or future-version payload: wallet reset to fresh
    }
    /// &lt;summary&gt;
    /// Wallet + profile persistence for G02 (task S2-T001, GDD s5 single-key JSON semantics).
    /// The payload travels through Sy.Core ISaveService: local storage is the only authority
    /// (red line 12 zero-server doctrine - no TCB, no cloud functions, no remote hot update).
    /// Version migration skeleton: a payload without the game version key is migrated by
    /// filling defaults for missing fields; a payload from a newer build degrades to a fresh
    /// wallet instead of crashing.
    /// &lt;/summary&gt;
    public static class G02SaveSystem
    {
        public const string Slot = "g02_main";
        public const string VersionKey = "g02v";
        public const int CurrentVersion = 1;
        /// &lt;summary&gt;Serialize the full profile into the single-key JSON payload.&lt;/summary&gt;
        public static string Serialize(SimWallet wallet, bool[] newbieFlags, int eraCount)
        {
            if (wallet == null) throw new ArgumentNullException(nameof(wallet));
            var data = new Dictionary&lt;string, object&gt;
            {
                { VersionKey, (long)CurrentVersion },
                { "money", (long)wallet.money },
                { "stars", (long)wallet.stars },
                { "era", (long)eraCount },
            };
            var phones = new List&lt;object&gt;();
            if (wallet.repairedPhones != null)
            {
                foreach (var id in wallet.repairedPhones) phones.Add(id);
            }
            data["repaired"] = phones;
            var stock = new Dictionary&lt;string, object&gt;();
            if (wallet.partStock != null)
            {
                foreach (var kv in wallet.partStock) stock[kv.Key] = (long)kv.Value;
            }
            data["stock"] = stock;
            var flags = new List&lt;object&gt;();
            if (newbieFlags != null)
            {
```

```
        foreach (var f in newbieFlags) flags.Add(f);
    }
    data["newbie"] = flags;
    return MiniJson.Serialize(data);
}
/// &lt;summary&gt;Write the profile through the save service (local authority).&lt;/summary&gt;
public static void Persist(ISaveService save, SimWallet wallet, bool[] newbieFlags, int eraCount,
    string slot = Slot)
{
    if (save == null) return;
    save.Save(slot, Serialize(wallet, newbieFlags, eraCount));
}
/// &lt;summary&gt;
/// GDD s5 fresh-wallet seeding (task S3-T005): the E1 starter stock (battery x2,
/// antenna, keypad) is applied on NewGame and CorruptedReset. Absolute-set per part:
/// a fresh wallet always starts from the catalog seed, never from stale leftovers.
/// A null catalog keeps the legacy empty-wallet behavior (test back-compat).
/// &lt;/summary&gt;
public static void SeedFreshStock(SimWallet wallet, GameCatalog catalog)
{
    if (wallet == null || catalog == null || catalog.InitialStock == null) return;
    foreach (var seed in catalog.InitialStock)
    {
        if (seed == null || string.IsNullOrEmpty(seed.id) || seed.count &lt;= 0) continue;
        wallet.partStock[seed.id] = seed.count;
    }
}
/// &lt;summary&gt;
/// Load the slot into the given wallet + newbie flags. Never throws on bad data:
/// corrupted or future-version payloads reset the wallet to fresh defaults.
/// Passing the catalog seeds the GDD s5 starter stock on fresh outcomes.
/// &lt;/summary&gt;
public static G02LoadStatus LoadInto(ISaveService save, SimWallet wallet, bool[] newbieFlags,
    out int eraCount, string slot = Slot, GameCatalog catalog = null)
{
    if (save == null) throw new ArgumentNullException(nameof(save));
    if (wallet == null) throw new ArgumentNullException(nameof(wallet));
    eraCount = 0;
    string raw;
    try
    {
        raw = save.Load(slot);
    }
    catch (Exception e)
    {
        // The service itself chokes on a corrupted payload - degrade, never crash the boot.
        Logger.Warn("G02.Save", "corrupt payload at service level, resetting slot: " + e.Message);
        return Fresh(wallet, newbieFlags, catalog);
    }
    if (string.IsNullOrEmpty(raw))
```

```
        {
            SeedFreshStock(wallet, catalog);
            return G02LoadStatus.NewGame;
        }
        Dictionary&lt;string, object&gt; data;
        try
        {
            data = MiniJson.Deserialize(raw) as Dictionary&lt;string, object&gt;;
        }
        catch (Exception e)
        {
            Logger.Warn("G02.Save", "corrupt payload, resetting slot: " + e.Message);
            return Fresh(wallet, newbieFlags, catalog);
        }
        if (data == null)
        {
            Logger.Warn("G02.Save", "non-dict payload, resetting slot");
            return Fresh(wallet, newbieFlags, catalog);
        }
        int version = ReadInt(data, VersionKey, 0);
        if (version &gt; CurrentVersion)
        {
            Logger.Warn("G02.Save", "future version " + version + " &gt; " + CurrentVersion + ", resetting slot");
            return Fresh(wallet, newbieFlags, catalog);
        }
        wallet.money = ReadInt(data, "money", 0);
        wallet.stars = ReadInt(data, "stars", 0);
        eraCount = ReadInt(data, "era", 0);
        wallet.repairedPhones = ReadStrings(data, "repaired");
        wallet.partStock = ReadStock(data, "stock");
        ReadFlags(data, "newbie", newbieFlags);
        return version &lt; CurrentVersion ? G02LoadStatus.Migrated : G02LoadStatus.Loaded;
    }
    static G02LoadStatus Fresh(SimWallet wallet, bool[] newbieFlags, GameCatalog catalog)
    {
        ResetWallet(wallet, newbieFlags);
        SeedFreshStock(wallet, catalog);
        return G02LoadStatus.CorruptedReset;
    }
    static void ResetWallet(SimWallet wallet, bool[] newbieFlags)
    {
        wallet.money = 0;
        wallet.stars = 0;
        wallet.repairedPhones = new List&lt;string&gt;();
        wallet.partStock = new Dictionary&lt;string, int&gt;();
        if (newbieFlags != null)
        {
            for (int i = 0; i &lt; newbieFlags.Length; i++) newbieFlags[i] = false;
        }
    }
```

```
static int ReadInt(Dictionary&lt;string, object&gt; data, string key, int fallback)
{
    if (!data.TryGetValue(key, out var value) || value == null) return fallback;
    try
    {
        return Convert.ToInt32(Convert.ToDouble(value));
    }
    catch
    {
        return fallback;
    }
}
static List&lt;string&gt; ReadStrings(Dictionary&lt;string, object&gt; data, string key)
{
    var list = new List&lt;string&gt;();
    if (data.TryGetValue(key, out var value) &amp;&amp; value is System.Collections.IEnumerable rows)
    {
        foreach (var row in rows)
        {
            if (row is string s &amp;&amp; !string.IsNullOrEmpty(s)) list.Add(s);
        }
    }
    return list;
}
static Dictionary&lt;string, int&gt; ReadStock(Dictionary&lt;string, object&gt; data, string key)
{
    var stock = new Dictionary&lt;string, int&gt;();
    if (data.TryGetValue(key, out var value) &amp;&amp; value is Dictionary&lt;string, object&gt; dict)
    {
        foreach (var kv in dict)
        {
            int amount;
            try { amount = Convert.ToInt32(Convert.ToDouble(kv.Value)); }
            catch { continue; }
            if (amount &gt; 0) stock[kv.Key] = amount;
        }
    }
    return stock;
}
static void ReadFlags(Dictionary&lt;string, object&gt; data, string key, bool[] newbieFlags)
{
    if (newbieFlags == null || !data.TryGetValue(key, out var value)) return;
    if (!(value is System.Collections.IEnumerable rows)) return;
    int i = 0;
    foreach (var row in rows)
    {
        if (i &gt;= newbieFlags.Length) break;
        newbieFlags[i++] = row is bool b &amp;&amp; b;
    }
}
```

```
    }
}
// G02Telemetry.cs
using System.Collections.Generic;
using Sy.Core;
using Sy.Telemetry;
using UnityEngine;
namespace Sy.G02
{
    /// &lt;summary&gt;
    /// Bridge from RepairSim bus events into the telemetry pipeline (task S1-T002):
    /// the repair funnel plus the newbie five steps. DebugLogSink is always attached;
    /// the Wx analytics sink joins at platform integration (zero-server doctrine:
    /// the TCB batch sink stays available but unregistered).
    /// &lt;/summary&gt;
    public sealed class G02TelemetryWire
    {
        // Newbie five steps (G02 first-session funnel, sim surface):
        // 1 accept_order  2 confirm_diagnosis  3 start_repair  4 first_step_done  5 first_settle.
        public const int NewbieStepCount = 5;
        const string NewbieAcceptOrder = "accept_order";
        const string NewbieConfirmDiagnosis = "confirm_diagnosis";
        const string NewbieStartRepair = "start_repair";
        const string NewbieFirstStepDone = "first_step_done";
        const string NewbieFirstSettle = "first_settle";
        readonly TelemetryService _telemetry;
        readonly bool[] _newbieFired = new bool[NewbieStepCount];
        public G02TelemetryWire(SyEventBus bus, TelemetryService telemetry)
        {
            _telemetry = telemetry ?? new TelemetryService();
            if (bus == null) return;
            bus.Sub&lt;SessionStartedEvent&gt;(OnSessionStarted);
            bus.Sub&lt;DiagnosisConfirmedEvent&gt;(OnDiagnosisConfirmed);
            bus.Sub&lt;RepairStartedEvent&gt;(OnRepairStarted);
            bus.Sub&lt;StepCompletedEvent&gt;(OnStepCompleted);
            bus.Sub&lt;SessionSettledEvent&gt;(OnSessionSettled);
            bus.Sub&lt;SessionClosedEvent&gt;(OnSessionClosed);
        }
        /// &lt;summary&gt;Standard G02 telemetry service: schema from Resources + debug sink.
        /// The Wx analytics sink joins when a reporter is available (S2-T002): an
        /// explicit param wins, otherwise the locator is consulted for the
        /// platform-registered Wx reporter on real builds; mock/editor stays
        /// debug-only. Zero-server red line 12: the TCB batch sink class stays
        /// available in the shared package but is never registered.&lt;/summary&gt;
        public static TelemetryService CreateService(IAnalyticsReporter reporter = null)
        {
            var telemetry = new TelemetryService();
            var asset = Resources.Load&lt;TextAsset&gt;("cfg_g02_events");
            if (asset != null) telemetry.LoadSchemaFromJson(asset.text);
            telemetry.AddSink(new DebugLogSink());
```

```
    if (reporter == null &amp;amp;&amp;amp; ServiceLocator.TryGet&amp;lt;IAnalyticsReporter&amp;gt;(out var fromLocator))
        reporter = fromLocator;
    if (reporter != null) telemetry.AddSink(new WxAnalyticsSink(reporter));
    return telemetry;
}
void OnSessionStarted(SessionStartedEvent evt)
{
    Track(G02FunnelPhases.SessionStart, Fields(
        "era", evt.eraId,
        "phone", evt.phoneId,
        "faults", evt.faultCount,
        "steps", evt.totalSteps,
        "partCost", evt.partChargeCost));
    FireNewbie(1, NewbieAcceptOrder);
}
void OnDiagnosisConfirmed(DiagnosisConfirmedEvent evt)
{
    Track(G02FunnelPhases.Diagnosis, null);
    FireNewbie(2, NewbieConfirmDiagnosis);
}
void OnRepairStarted(RepairStartedEvent evt)
{
    Track(G02FunnelPhases.RepairStart, null);
    FireNewbie(3, NewbieStartRepair);
}
void OnStepCompleted(StepCompletedEvent evt)
{
    Track(G02FunnelPhases.Step, Fields(
        "stepIndex", evt.stepIndex,
        "perfect", evt.perfect,
        "mistakes", evt.mistakes));
    if (evt.stepIndex == 0) FireNewbie(4, NewbieFirstStepDone);
}
void OnSessionSettled(SessionSettledEvent evt)
{
    Track(G02FunnelPhases.Settle, Fields(
        "phone", evt.phoneId,
        "perfect", evt.perfect,
        "mistakes", evt.mistakes,
        "pay", evt.pay,
        "stars", evt.starsGained));
    FireNewbie(5, NewbieFirstSettle);
}
void OnSessionClosed(SessionClosedEvent evt)
{
    Track(G02FunnelPhases.SessionClose, null);
}
/// &amp;lt;summary&amp;gt;First-session one-shot per step; persistence lands with the save slice.&amp;lt;/summary&amp;gt;
void FireNewbie(int step, string name)
{
```

```
            bool same = first.Length == other.Length;
            if (same)
            {
                for (int s = 0; s &lt; first.Length; s++)
                {
                    if (Math.Abs(first[s] - other[s]) &gt; 1e-4)
                    {
                        same = false;
                        break;
                    }
                }
            }
            Assert.That(same, Is.False, AllClips[i] + " must differ from " + AllClips[0]);
        }
    }
    [Test]
    public void Loudness_PeakBoundedAndMistakeToneIsWeak()
    {
        foreach (var id in AllClips)
        {
            var data = G02Sfx.Synth(id);
            Assert.That(Peak(data), Is.LessThanOrEqualTo(0.95), id + " peak must stay under the clip ceiling");
            Assert.That(Rms(data), Is.InRange(0.02, 0.5), id + " rms must sit inside the loudness family");
        }
        Assert.That(Peak(G02Sfx.Synth(G02Sfx.MissTone)),
            Is.LessThan(Peak(G02Sfx.Synth(G02Sfx.ScrewTap))),
            "the mistake tone must be weaker than the core feedback trio");
    }
    [Test]
    public void Leitmotif_InsideThreeSecondHummableRedLine()
    {
        var data = G02Sfx.Synth(G02Sfx.Leitmotif);
        double seconds = (double)data.Length / G02Sfx.SampleRate;
        Assert.That(seconds, Is.GreaterThanOrEqualTo(1.0), "hook needs enough room to be hummable");
        Assert.That(seconds, Is.LessThanOrEqualTo(3.0), "signature hook must stay inside the 3s red line");
        // wind-up triple rise: three note regions must carry real energy
        int noteSamples = (int)(G02Sfx.SampleRate * 0.18);
        for (int n = 0; n &lt; 3; n++)
        {
            double regionPeak = 0;
            for (int s = 0; s &lt; noteSamples; s++)
            {
                double a = Math.Abs(data[n * noteSamples + s]);
                if (a &gt; regionPeak) regionPeak = a;
            }
            Assert.That(regionPeak, Is.GreaterThan(0.3), "note " + (n + 1) + " of the rise must be audible");
        }
        // breath before the bell: the gap region must be quiet (discrete contour)
        int gapStart = (int)(G02Sfx.SampleRate * 0.56);
        int gapEnd = (int)(G02Sfx.SampleRate * 0.62);
```

```
            for (int s = gapStart; s &lt; gapEnd; s++)
            {
                Assert.That(Math.Abs(data[s]), Is.LessThan(0.05), "breath gap must be quiet at sample " + s);
            }
        }
        [Test]
        public void Clip_FactoryWrapsSynthSamples()
        {
            var clip = G02Sfx.Clip(G02Sfx.SettleBell);
            Assert.That(clip, Is.Not.Null, "factory must create an AudioClip");
            Assert.That(clip.frequency, Is.EqualTo(G02Sfx.SampleRate));
            Assert.That(clip.samples, Is.EqualTo(G02Sfx.Synth(G02Sfx.SettleBell).Length));
        }
    }
}
// G02ShareCardTests.cs
using System.IO;
using NUnit.Framework;
using Sy.G02;
using Sy.Platform;
using UnityEngine;
namespace Sy.G02.Tests
{
    // S3-T004 acceptance: View parity + the pixel-readback gold canary (RT three
    // laws #3). The canary samples known-solid element centers through the same
    // layout solver the composer uses, so a blank or clipped card fails loudly
    // even though size and byte budget would have passed (F4 lesson).
    public class G02ShareCardTests
    {
        G02ShareCardComposer.ComposeResult _result;
        G02ShareCardComposer.ComposeResult ComposeSample()
        {
            G02Text.Load();
            int layer = G02ShareCardComposer.ResolveLayer();
            Assert.GreaterOrEqual(layer, 0, "G02ShareCard layer must exist in TagManager");
            var info = new G02ShareCardInfo
            {
                PhoneName = "TALKBLOCK 1998",
                Fee = 128,
                StarsGained = 3,
                Perfect = true,
                Quip = "test quip",
            };
            var result = G02ShareCardComposer.Compose(info, layer);
            Assert.IsNotNull(result, "compose must succeed with a valid layer");
            _result = result;
            return result;
        }
        [TearDown]
        public void TearDown()
```

```
{
    G02ShareCardComposer.Release(_result);
    _result = null;
}
[Test]
public void Layout_Maps_Reference_Center_To_Pixel_Center()
{
    var p = G02ShareCardLayout.ToPixel(Vector2.zero);
    Assert.AreEqual(G02ShareCardLayout.RtW * 0.5f, p.x, 0.001f);
    Assert.AreEqual(G02ShareCardLayout.RtH * 0.5f, p.y, 0.001f);
    var corner = G02ShareCardLayout.ToPixel(new Vector2(-540f, -960f));
    Assert.AreEqual(0f, corner.x, 0.001f);
    Assert.AreEqual(0f, corner.y, 0.001f);
}
[Test]
public void Compose_Produces_Png_With_Exact_Size()
{
    var result = ComposeSample();
    Assert.IsNotNull(result.Png);
    Assert.Greater(result.Png.Length, 0, "PNG bytes must exist");
    Assert.AreEqual(G02ShareCardLayout.RtW, result.Width);
    Assert.AreEqual(G02ShareCardLayout.RtH, result.Height);
    Assert.AreEqual(G02ShareCardLayout.RtW, result.Texture.width);
    Assert.AreEqual(G02ShareCardLayout.RtH, result.Texture.height);
}
[Test]
public void PixelReadback_Canary_Solid_Samples_Match()
{
    var result = ComposeSample();
    foreach (var sample in G02ShareCardLayout.SolidSamples())
    {
        Color got = G02ShareCardComposer.SampleAt(result, sample.Point);
        Assert.LessOrEqual(Mathf.Abs(got.r - sample.Color.r), 0.06f,
            sample.Name + " red channel drift");
        Assert.LessOrEqual(Mathf.Abs(got.g - sample.Color.g), 0.06f,
            sample.Name + " green channel drift");
        Assert.LessOrEqual(Mathf.Abs(got.b - sample.Color.b), 0.06f,
            sample.Name + " blue channel drift");
    }
}
[Test]
public void Layer_Traversal_Covers_Every_Card_Object()
{
    var result = ComposeSample();
    int layer = G02ShareCardComposer.ResolveLayer();
    Assert.AreEqual(1 &lt;&lt; layer, result.Cam.cullingMask,
        "camera must see only the share-card layer");
    AssertAllOnLayer(result.Rig.transform, layer);
}
static void AssertAllOnLayer(Transform t, int layer)
```

```
{
    Assert.AreEqual(layer, t.gameObject.layer,
        "object left behind on a wrong layer: " + t.name);
    for (int i = 0; i &lt; t.childCount; i++) AssertAllOnLayer(t.GetChild(i), layer);
}
[Test]
public void BuildPayload_Carries_Title_Query_And_Local_Path()
{
    G02Text.Load();
    var info = new G02ShareCardInfo { PhoneName = "TALKBLOCK 1998", Fee = 128 };
    var payload = G02ShareCardComposer.BuildPayload(info, "C:/tmp/g02_share_card.png");
    StringAssert.Contains("TALKBLOCK 1998", payload.Title,
        "share title must carry the repaired phone (bragging hook)");
    StringAssert.DoesNotContain("{phone}", payload.Title, "template must be resolved");
    Assert.AreEqual("src=share", payload.Query, "zero-server launch attribution");
    Assert.AreEqual("C:/tmp/g02_share_card.png", payload.ImageUrl, "local path passes through");
}
[Test]
public void FromSettle_Mirrors_Settle_Info()
{
    var settle = new G02SettleInfo
    {
        phoneName = "BRICK 3210",
        pay = 88,
        starsGained = 2,
        perfect = false,
        quip = "ok quip",
    };
    var card = G02ShareCardInfo.FromSettle(settle);
    Assert.AreEqual("BRICK 3210", card.PhoneName);
    Assert.AreEqual(88, card.Fee);
    Assert.AreEqual(2, card.StarsGained);
    Assert.IsFalse(card.Perfect);
    Assert.AreEqual("ok quip", card.Quip);
}
// S4-T004 mock face: the full brag-share chain through the platform bridge.
// The real-device leg (WX share with a local imageUrl) stays blocked_on the
// release account (U014); this face proves the bridge contract plus the
// red-line-12 local-PNG semantics every channel must honor.
[Test]
public void MockShare_FullChain_LandsPngLocally_AndRecordsLocalUrl()
{
    var result = ComposeSample();
    string path = G02ShareCardComposer.WritePng(result.Png);
    Assert.IsNotEmpty(path, "compose must land a real local PNG path");
    Assert.IsTrue(File.Exists(path), "share PNG must exist on disk before Share is called");
    var info = new G02ShareCardInfo { PhoneName = "TALKBLOCK 1998", Fee = 128 };
    var payload = G02ShareCardComposer.BuildPayload(info, path);
    var mock = new MockPlatformService();
    mock.Share(payload);
```

```
            Assert.AreEqual(1, mock.Shares.Count, "mock bridge must record exactly one share");
            var sent = mock.Shares[0];
            Assert.AreEqual(path, sent.ImageUrl, "bridge must pass the local path through untouched");
            StringAssert.DoesNotContain("http", sent.ImageUrl,
                "red line 12: share image stays a local path, never a URL");
            StringAssert.Contains("TALKBLOCK 1998", sent.Title,
                "share title carries the repaired phone (bragging hook)");
            Assert.AreEqual("src=share", sent.Query, "zero-server launch attribution");
        }
    }
}
// G02StepShowTests.cs
using NUnit.Framework;
using Sy.G02;
namespace Sy.G02.Tests
{
    /// &lt;summary&gt;
    /// Per-step instant numeric feedback tests (task S2-T004 slice B, GAP-D second half):
    /// fee-preview display parity with the Economy settlement, mistake-hint parity with
    /// the projected reward pct, and pop-window math properties.
    /// &lt;/summary&gt;
    public class G02StepShowTests
    {
        GameCatalog _catalog;
        G02Presenter _presenter;
        [SetUp]
        public void SetUp()
        {
            _catalog = GameCatalog.Load();
            Assert.That(_catalog, Is.Not.Null, "catalog must load from Resources in batchmode");
            G02Text.Load();
            _presenter = new G02Presenter(_catalog, new RepairSim(_catalog));
        }
        [Test]
        public void FeePreviewParityWithEconomyAndSettlePay()
        {
            _presenter.AcceptNext(321);
            _presenter.ConfirmDiagnosis();
            _presenter.PickDiagnosis(0);
            _presenter.StartRepair();
            var session = _presenter.Sim.Session;
            _presenter.DoStep(true);
            // Mid-repair the preview delegates to Economy with the live session state
            // (a quick-boot hidden fault is not on the bench yet).
            int midPreview = G02StepShow.FeePreview(
                session.order.eraId, session.faults.Length, session.mistakes, _catalog);
            Assert.That(midPreview, Is.EqualTo(Economy.FeeFor(
                session.order.eraId, session.faults.Length, session.mistakes, _catalog)),
                "preview must delegate to Economy with the live session state");
            while (!_presenter.BuildRepair().done) _presenter.DoStep(true);
```

```
    _presenter.Finish();
    Assert.That(_presenter.Phase, Is.EqualTo(SimPhase.Settle));
    // U153-D01 probe-aware realign: once the quick-boot rework has surfaced, the
    // preview applies the rework discount, so the fee projected after the final
    // step equals the settle payment.
    int finalPreview = G02StepShow.FeePreview(
        session.order.eraId, session.faults.Length, session.mistakes, _catalog,
        session.reworkTriggered, session.tipPct);
    Assert.That(_presenter.BuildSettle().pay, Is.EqualTo(finalPreview),
        "probe-aware projected fee after the final step equals the settle payment");
}
[Test]
public void MistakeHintParityWithProjectedRewardPct()
{
    Assert.That(G02StepShow.PctAfterMistake(0, _catalog), Is.EqualTo(Economy.RewardPct(1, _catalog)));
    Assert.That(G02StepShow.PctAfterMistake(2, _catalog), Is.EqualTo(Economy.RewardPct(3, _catalog)));
    _presenter.AcceptNext(77);
    _presenter.ConfirmDiagnosis();
    _presenter.PickDiagnosis(1);
    _presenter.StartRepair();
    var session = _presenter.Sim.Session;
    int hintPct = G02StepShow.PctAfterMistake(session.mistakes, _catalog);
    _presenter.DoStep(false); // the step lands imperfect exactly as the hint projected
    while (!_presenter.BuildRepair().done) _presenter.DoStep(true);
    _presenter.Finish();
    Assert.That(_presenter.BuildSettle().rewardPct, Is.EqualTo(hintPct),
        "hint pct at the mistake moment equals the settle reward pct");
}
[Test]
public void FeePreviewRespectsMistakePenalty()
{
    _presenter.AcceptNext(55);
    _presenter.ConfirmDiagnosis();
    _presenter.PickDiagnosis(2);
    _presenter.StartRepair();
    var session = _presenter.Sim.Session;
    _presenter.DoStep(true);
    int clean = G02StepShow.FeePreview(
        session.order.eraId, session.faults.Length, session.mistakes, _catalog);
    Assert.That(clean, Is.EqualTo(Economy.FeeFor(
        session.order.eraId, session.faults.Length, 0, _catalog)));
    if (session.totalSteps &gt;= 2)
    {
        _presenter.DoStep(false);
        int penalized = G02StepShow.FeePreview(
            session.order.eraId, session.faults.Length, session.mistakes, _catalog);
        Assert.That(penalized, Is.EqualTo(Economy.FeeFor(
            session.order.eraId, session.faults.Length, 1, _catalog)));
        Assert.That(penalized, Is.LessThanOrEqualTo(clean),
            "a mistake can never raise the projected fee");
```

```
            }
        }
        [Test]
        public void PopWindowMathStaysInsideBounds()
        {
            foreach (float window in new[] { G02StepShow.PopSeconds, G02StepShow.HintSeconds })
            {
                Assert.That(G02StepShow.RiseOffset(0f, window), Is.EqualTo(0f));
                Assert.That(G02StepShow.RiseOffset(window, window), Is.EqualTo(G02StepShow.RiseUnits));
                Assert.That(G02StepShow.RiseOffset(window + 5f, window), Is.EqualTo(G02StepShow.RiseUnits));
                Assert.That(G02StepShow.Alpha(0f, window), Is.EqualTo(1f));
                Assert.That(G02StepShow.Alpha(window, window), Is.EqualTo(0f));
                Assert.That(G02StepShow.PopScale(0f, window), Is.EqualTo(1f));
                Assert.That(G02StepShow.PopScale(window, window), Is.EqualTo(1f));
                Assert.That(G02StepShow.PopScale(window + 5f, window), Is.EqualTo(1f));
                float prevRise = 0f;
                float peakScale = 0f;
                for (int i = 1; i &lt; 20; i++)
                {
                    float t = i * window / 20f;
                    float rise = G02StepShow.RiseOffset(t, window);
                    Assert.That(rise &gt;= prevRise, Is.True, "rise must be monotonic non-decreasing");
                    prevRise = rise;
                    peakScale = UnityEngine.Mathf.Max(peakScale, G02StepShow.PopScale(t, window));
                    Assert.That(G02StepShow.Alpha(t, window), Is.InRange(0f, 1f));
                }
                Assert.That(peakScale, Is.GreaterThan(1f),
                    "pop must visibly overshoot inside the window");
                Assert.That(peakScale, Is.LessThanOrEqualTo(1.25f + 0.001f),
                    "pop overshoot stays restrained");
            }
        }
        [Test]
        public void PopTextCarriesValuesAndI18nKeys()
        {
            Assert.That(G02StepShow.FeePopText(G02Text.Get("ui.step.feepreview"), 120),
                Is.EqualTo(G02Text.Get("ui.step.feepreview") + " +120"));
            Assert.That(G02StepShow.PctHintText(G02Text.Get("ui.settle.pct"), 90),
                Is.EqualTo(G02Text.Get("ui.settle.pct") + " 90%"));
            Assert.That(G02Text.Get("ui.step.feepreview"), Is.Not.EqualTo("ui.step.feepreview"),
                "fee preview key must exist in the i18n table");
        }
    }
}
// G02TelemetryTests.cs
using System.Collections.Generic;
using NUnit.Framework;
using Sy.Core;
using Sy.G02;
using Sy.Telemetry;
```

```
namespace Sy.G02.Tests
{
    /// &lt;summary&gt;
    /// Event bus + telemetry wiring tests (task S1-T002): bus event sequence for a full
    /// repair loop, funnel sink payloads with clean schema validation, and the newbie
    /// five-step one-shot behavior.
    /// &lt;/summary&gt;
    public class G02TelemetryTests
    {
        GameCatalog _catalog;
        [SetUp]
        public void SetUp()
        {
            _catalog = GameCatalog.Load();
            Assert.That(_catalog, Is.Not.Null, "catalog must load from Resources in batchmode");
            G02Text.Load();
        }
        /// &lt;summary&gt;Records every tracked event (id + field set) for assertions.&lt;/summary&gt;
        sealed class CaptureSink : TelemetryService.ISink
        {
            public readonly List&lt;string&gt; Ids = new List&lt;string&gt;();
            public readonly List&lt;Dictionary&lt;string, string&gt;&gt; FieldSets = new List&lt;Dictionary&lt;string, s
            public void Write(string evtId, Dictionary&lt;string, string&gt; fields)
            {
                Ids.Add(evtId);
                FieldSets.Add(new Dictionary&lt;string, string&gt;(fields));
            }
        }
        static void RunFullLoop(RepairSim sim, int seed, int imperfectSteps)
        {
            sim.AcceptNextCustomer(seed);
            sim.ConfirmDiagnosis();
            sim.StartRepair();
            int done = 0;
            while (!sim.Session.RepairDone)
            {
                sim.CompleteStep(done &gt;= imperfectSteps);
                done++;
            }
            sim.FinishRepair();
            sim.ReturnToMenu();
        }
        [Test]
        public void Bus_FullLoop_PublishesExactFunnelSequence()
        {
            var bus = new SyEventBus();
            var log = new List&lt;string&gt;();
            bus.Sub&lt;SessionStartedEvent&gt;(e =&gt; log.Add("started " + e.phoneId + " steps=" + e.totalSteps));
            bus.Sub&lt;DiagnosisConfirmedEvent&gt;(_ =&gt; log.Add("diagnosis"));
            bus.Sub&lt;RepairStartedEvent&gt;(_ =&gt; log.Add("repair_start"));
```

```
    bus.Sub&lt;StepCompletedEvent&gt;(e =&gt; log.Add("step " + e.stepIndex + " perfect=" + e.perfect));
    bus.Sub&lt;SessionSettledEvent&gt;(e =&gt; log.Add("settled pay=" + e.pay));
    bus.Sub&lt;SessionClosedEvent&gt;(_ =&gt; log.Add("closed"));
    var sim = new RepairSim(_catalog, new SimWallet { money = 0, stars = 0 }, bus);
    sim.AcceptNextCustomer(301);
    string phoneId = sim.Session.order.phoneId;
    int totalSteps = sim.Session.totalSteps;
    Assert.That(totalSteps, Is.GreaterThan(0));
    sim.ConfirmDiagnosis();
    sim.StartRepair();
    for (int i = 0; i &lt; totalSteps; i++) sim.CompleteStep(i &gt; 0); // first step fails, rest perfect
    var result = sim.FinishRepair();
    sim.ReturnToMenu();
    var expected = new List&lt;string&gt;
    {
        "started " + phoneId + " steps=" + totalSteps,
        "diagnosis",
        "repair_start",
    };
    for (int i = 0; i &lt; totalSteps; i++) expected.Add("step " + i + " perfect=" + (i &gt; 0));
    expected.Add("settled pay=" + result.pay);
    expected.Add("closed");
    Assert.That(log, Is.EqualTo(expected), "bus must receive the exact funnel event sequence");
}
[Test]
public void Wire_FunnelTracks_ReachSinkWithCleanSchema()
{
    var bus = new SyEventBus();
    var telemetry = G02TelemetryWire.CreateService();
    Assert.That(telemetry.SchemaEventCount, Is.EqualTo(2), "cfg_g02_events.json defines two events");
    var sink = new CaptureSink();
    telemetry.AddSink(sink);
    var wire = new G02TelemetryWire(bus, telemetry);
    Assert.That(wire, Is.Not.Null, "wire must construct and subscribe");
    var sim = new RepairSim(_catalog, new SimWallet { money = 0, stars = 0 }, bus);
    sim.AcceptNextCustomer(401);
    var order = sim.Session.order;
    int totalSteps = sim.Session.totalSteps;
    sim.ConfirmDiagnosis();
    sim.StartRepair();
    int mistakes = 0;
    for (int i = 0; i &lt; totalSteps; i++)
    {
        bool perfect = i &gt; 0;
        if (!perfect) mistakes++;
        sim.CompleteStep(perfect);
    }
    var result = sim.FinishRepair();
    sim.ReturnToMenu();
    var phases = new List&lt;string&gt;();
```

```
        for (int i = 0; i &lt; sink.Ids.Count; i++)
        {
            var fields = sink.FieldSets[i];
            Assert.That(fields.ContainsKey("__missing"), Is.False, "missing required field on " + sink.Ids[i]);
            Assert.That(fields.ContainsKey("__badtype"), Is.False, "type mismatch on " + sink.Ids[i]);
            if (sink.Ids[i] != G02EventIds.RepairFunnel) continue;
            phases.Add(fields["phase"]);
            if (fields["phase"] == G02FunnelPhases.SessionStart)
            {
                Assert.That(fields["era"], Is.EqualTo("" + order.eraId), "session_start era");
                Assert.That(fields["phone"], Is.EqualTo(order.phoneId), "session_start phone");
                Assert.That(fields["faults"], Is.EqualTo("" + order.faultIds.Length), "session_start faults");
            }
            else if (fields["phase"] == G02FunnelPhases.Settle)
            {
                Assert.That(fields["pay"], Is.EqualTo("" + result.pay), "settle pay");
                Assert.That(fields["stars"], Is.EqualTo("" + result.starsGained), "settle stars");
                Assert.That(fields["perfect"], Is.EqualTo("" + result.perfect), "settle perfect");
                Assert.That(fields["mistakes"], Is.EqualTo("" + mistakes), "settle mistakes");
            }
        }
        var expectedPhases = new List&lt;string&gt;
        {
            G02FunnelPhases.SessionStart,
            G02FunnelPhases.Diagnosis,
            G02FunnelPhases.RepairStart,
        };
        for (int i = 0; i &lt; totalSteps; i++) expectedPhases.Add(G02FunnelPhases.Step);
        expectedPhases.Add(G02FunnelPhases.Settle);
        expectedPhases.Add(G02FunnelPhases.SessionClose);
        Assert.That(phases, Is.EqualTo(expectedPhases), "funnel phases must arrive in loop order");
    }
    [Test]
    public void Wire_NewbieFiveSteps_FireOncePerStep_NotOnSecondLoop()
    {
        var bus = new SyEventBus();
        var telemetry = G02TelemetryWire.CreateService();
        var sink = new CaptureSink();
        telemetry.AddSink(sink);
        var wire = new G02TelemetryWire(bus, telemetry);
        var sim = new RepairSim(_catalog, new SimWallet { money = 0, stars = 0 }, bus);
        RunFullLoop(sim, 501, 0);
        RunFullLoop(sim, 502, 0);
        var newbie = new List&lt;int&gt;();
        var newbieNames = new List&lt;string&gt;();
        for (int i = 0; i &lt; sink.Ids.Count; i++)
        {
            if (sink.Ids[i] != G02EventIds.NewbieStep) continue;
            newbie.Add(int.Parse(sink.FieldSets[i]["step"]));
            newbieNames.Add(sink.FieldSets[i]["name"]);
```

```
            }
            Assert.That(newbie, Is.EqualTo(new List&lt;int&gt; { 1, 2, 3, 4, 5 }),
                "exactly five newbie steps, once each, across both loops");
            Assert.That(newbieNames, Is.EqualTo(new List&lt;string&gt;
            {
                "accept_order", "confirm_diagnosis", "start_repair", "first_step_done", "first_settle",
            }));
        }
    }
}
// G02ViewTests.cs
using System;
using System.Collections.Generic;
using NUnit.Framework;
using Sy.G02;
namespace Sy.G02.Tests
{
    /// &lt;summary&gt;
    /// Presenter view-model tests (task S1-T003 slice A): desk text resolution, any-choice
    /// diagnosis reveal, repair step list parity, settle parity and menu stats.
    /// &lt;/summary&gt;
    public class G02ViewTests
    {
        GameCatalog _catalog;
        G02Presenter _presenter;
        [SetUp]
        public void SetUp()
        {
            _catalog = GameCatalog.Load();
            Assert.That(_catalog, Is.Not.Null, "catalog must load from Resources in batchmode");
            G02Text.Load();
            _presenter = new G02Presenter(_catalog, new RepairSim(_catalog));
        }
        [Test]
        public void DeskInfoResolvesCatalogText()
        {
            _presenter.AcceptNext(12345);
            Assert.That(_presenter.Phase, Is.EqualTo(SimPhase.Desk));
            var info = _presenter.BuildDesk();
            var order = _presenter.Sim.Session.order;
            Assert.That(info.customerName, Is.EqualTo(G02Text.Get(order.customerNameKey)));
            Assert.That(info.openLine, Is.EqualTo(G02Text.Get(order.openLineKey)));
            Assert.That(info.phoneName, Is.Not.Empty);
            Assert.That(info.eraYears, Is.Not.Empty);
            int expected = 0;
            foreach (var fault in _presenter.Sim.Session.faults)
            {
                expected += fault.symptomKeys.Length;
            }
            Assert.That(info.symptoms.Length, Is.EqualTo(expected));
```

```
    Assert.That(info.money, Is.EqualTo(_presenter.Sim.Wallet.money));
}
[Test]
public void DiagnosisAnyOptionRevealsFaultsAndGuardsPhase()
{
    _presenter.AcceptNext(9);
    _presenter.ConfirmDiagnosis();
    var info = _presenter.BuildDiagnosis();
    Assert.That(info.revealed, Is.False);
    Assert.That(info.options.Length, Is.EqualTo(G02Presenter.DiagnosisOptionCount));
    foreach (var option in info.options) Assert.That(option, Is.Not.Empty);
    Assert.That(info.faults, Is.Not.Null);
    _presenter.PickDiagnosis(2);
    info = _presenter.BuildDiagnosis();
    Assert.That(info.revealed, Is.True);
    var faults = _presenter.Sim.Session.faults;
    Assert.That(info.faults.Length, Is.EqualTo(faults.Length));
    for (int i = 0; i &lt; faults.Length; i++)
    {
        Assert.That(info.faults[i].name, Is.EqualTo(G02Text.Get(faults[i].nameKey)));
    }
    // U153-D01 realign: a second pick is a double-resolve; the idempotence
    // guard throws and the reveal state stays put.
    Assert.Throws&lt;InvalidOperationException&gt;(
        delegate { _presenter.PickDiagnosis(1); });
    Assert.That(_presenter.BuildDiagnosis().revealed, Is.True);
    // Picking outside the Diagnosis phase throws like the sim guards.
    _presenter.StartRepair();
    Assert.Throws&lt;InvalidOperationException&gt;(
        delegate { _presenter.PickDiagnosis(0); });
}
[Test]
public void RepairStepListParityWithFaultSteps()
{
    _presenter.AcceptNext(7);
    _presenter.ConfirmDiagnosis();
    _presenter.PickDiagnosis(0);
    _presenter.StartRepair();
    var info = _presenter.BuildRepair();
    Assert.That(info.done, Is.False);
    Assert.That(info.index, Is.EqualTo(0));
    Assert.That(info.current, Is.Not.Empty);
    var expected = new List&lt;string&gt;();
    foreach (var fault in _presenter.Sim.Session.faults)
    {
        foreach (var step in fault.steps) expected.Add(G02Text.Get(_catalog.StepNameKey(step)));
    }
    Assert.That(info.steps, Is.EqualTo(expected));
    Assert.That(info.total, Is.EqualTo(_presenter.Sim.Session.totalSteps));
    while (!info.done)
```

```
        {
            _presenter.DoStep(true);
            info = _presenter.BuildRepair();
        }
        Assert.That(info.done, Is.True);
        Assert.That(info.index, Is.EqualTo(info.total));
        Assert.That(info.current, Is.Empty);
        Assert.That(info.mistakes, Is.EqualTo(0), "slice A steps are always perfect");
    }
    [Test]
    public void SettleInfoParityWithSimResult()
    {
        _presenter.AcceptNext(777);
        _presenter.ConfirmDiagnosis();
        _presenter.PickDiagnosis(1);
        _presenter.StartRepair();
        while (!_presenter.BuildRepair().done) _presenter.DoStep(true);
        _presenter.Finish();
        Assert.That(_presenter.Phase, Is.EqualTo(SimPhase.Settle));
        var info = _presenter.BuildSettle();
        var result = _presenter.Sim.LastResult;
        Assert.That(info.pay, Is.EqualTo(result.pay));
        Assert.That(info.rewardPct, Is.EqualTo(result.rewardPct));
        Assert.That(info.starsGained, Is.EqualTo(result.starsGained));
        Assert.That(info.perfect, Is.True);
        Assert.That(info.quip, Is.EqualTo(G02Text.Get(result.quipKey)));
        var phone = _catalog.Phone(_presenter.Sim.Session.order.phoneId);
        Assert.That(info.trivia, Is.EqualTo(G02Text.Get(phone.triviaKey)));
        Assert.That(info.phoneName, Is.EqualTo(G02Text.Get(phone.nameKey)));
        Assert.That(info.money, Is.EqualTo(_presenter.Sim.Wallet.money));
        Assert.That(info.stars, Is.EqualTo(_presenter.Sim.Wallet.stars));
    }
    [Test]
    public void MenuInfoReflectsWalletAndEras()
    {
        _presenter.AcceptNext(42);
        _presenter.ConfirmDiagnosis();
        _presenter.PickDiagnosis(2);
        _presenter.StartRepair();
        while (!_presenter.BuildRepair().done) _presenter.DoStep(true);
        _presenter.Finish();
        _presenter.Again();
        Assert.That(_presenter.Phase, Is.EqualTo(SimPhase.Menu));
        var info = _presenter.BuildMenu();
        Assert.That(info.repairedCount, Is.EqualTo(1));
        Assert.That(info.unlockedEras, Is.EqualTo(_catalog.UnlockedEraCount(_presenter.Sim.Wallet.stars)));
        Assert.That(info.totalEras, Is.EqualTo(_catalog.Eras.Length));
        Assert.That(info.money, Is.EqualTo(_presenter.Sim.Wallet.money));
        Assert.That(info.stars, Is.EqualTo(_presenter.Sim.Wallet.stars));
    }
```

```
    }
}
// RepairSimTests.cs
using System.Collections.Generic;
using System.Linq;
using NUnit.Framework;
using Sy.G02;
namespace Sy.G02.Tests
{
    /// &lt;summary&gt;
    /// RepairSim state-machine tests (task S1-T001 slice B): phase flow, part charging,
    /// settle economy parity, determinism, era cap, museum tracking and quip picks.
    /// &lt;/summary&gt;
    public class RepairSimTests
    {
        GameCatalog _catalog;
        [SetUp]
        public void SetUp()
        {
            _catalog = GameCatalog.Load();
            Assert.That(_catalog, Is.Not.Null, "catalog must load from Resources in batchmode");
            G02Text.Load();
        }
        /// &lt;summary&gt;Drive the sim from Desk through settle back to Menu.&lt;/summary&gt;
        static SimSettleResult WalkDeskToMenu(RepairSim sim, int imperfectSteps)
        {
            sim.ConfirmDiagnosis();
            Assert.That(sim.Phase, Is.EqualTo(SimPhase.Diagnosis));
            sim.StartRepair();
            Assert.That(sim.Phase, Is.EqualTo(SimPhase.Repair));
            int done = 0;
            while (!sim.Session.RepairDone)
            {
                // First imperfectSteps steps fail (count as mistakes), the rest are perfect.
                sim.CompleteStep(done &gt;= imperfectSteps);
                done++;
            }
            var result = sim.FinishRepair();
            Assert.That(sim.Phase, Is.EqualTo(SimPhase.Settle));
            sim.ReturnToMenu();
            Assert.That(sim.Phase, Is.EqualTo(SimPhase.Menu));
            Assert.That(sim.Session, Is.Null, "session must close on ReturnToMenu");
            return result;
        }
        /// &lt;summary&gt;Full loop including the accept step.&lt;/summary&gt;
        static SimSettleResult RunFullLoop(RepairSim sim, int seed, int imperfectSteps)
        {
            sim.AcceptNextCustomer(seed);
            Assert.That(sim.Phase, Is.EqualTo(SimPhase.Desk), "accept should move to Desk");
            Assert.That(sim.Session, Is.Not.Null, "session must exist at Desk");
```

```
    return WalkDeskToMenu(sim, imperfectSteps);
}
/// &lt;summary&gt;Parts auto-charged on accept for an order (mirrors sim ConsumePartsOnAccept).&lt;/summary&gt;
static int ChargeFor(RepairOrder order, GameCatalog catalog)
{
    int total = 0;
    foreach (var id in order.faultIds)
    {
        var fault = catalog.Fault(id);
        if (fault == null || string.IsNullOrEmpty(fault.partId)) continue;
        var part = catalog.Part(fault.partId);
        int cost = part != null &amp;&amp; order.eraId &gt;= 0 &amp;&amp; order.eraId &lt; part.costs.Length
            ? part.costs[order.eraId]
            : 0;
        if (cost &gt; 0) total += cost;
    }
    return total;
}
/// &lt;summary&gt;U153-D02: credit auto-charge on accept mirrors sim ConsumePartsOnAccept
/// (catalog cost + 110% markup, integer math per part).&lt;/summary&gt;
static int CreditChargeFor(RepairOrder order, GameCatalog catalog)
{
    int total = 0;
    foreach (var id in order.faultIds)
    {
        var fault = catalog.Fault(id);
        if (fault == null || string.IsNullOrEmpty(fault.partId)) continue;
        var part = catalog.Part(fault.partId);
        int cost = part != null &amp;&amp; order.eraId &gt;= 0 &amp;&amp; order.eraId &lt; part.costs.Length
            ? part.costs[order.eraId]
            : 0;
        if (cost &gt; 0) total += cost * RepairSim.CreditMarkupPct / 100;
    }
    return total;
}
[Test]
public void Accept_FromMenu_RollsValidOrderForUnlockedEra()
{
    var sim = new RepairSim(_catalog);
    sim.AcceptNextCustomer(7);
    var order = sim.Session.order;
    Assert.That(order.eraId, Is.InRange(0, sim.UnlockedEraCount - 1));
    var phone = _catalog.Phone(order.phoneId);
    Assert.That(phone, Is.Not.Null, "order phone must resolve");
    Assert.That(phone.era, Is.EqualTo(order.eraId));
    Assert.That(order.faultIds.Length, Is.InRange(1, 2));
    int expectedSteps = 0;
    foreach (var id in order.faultIds)
    {
        var fault = _catalog.Fault(id);
```

```
            Assert.That(fault, Is.Not.Null, "order fault must resolve");
            Assert.That(_catalog.FaultFitsEra(fault, order.eraId), Is.True,
                "fault must fit the order era");
            expectedSteps += fault.steps.Length;
        }
        Assert.That(sim.Session.totalSteps, Is.EqualTo(expectedSteps));
        Assert.That(sim.Session.stepsDone, Is.EqualTo(0));
    }
    [Test]
    public void FullLoop_PerfectRepair_PaysEconomyFee_AndGainsTwoStars()
    {
        var wallet = new SimWallet { money = 500, stars = 0 };
        var sim = new RepairSim(_catalog, wallet);
        var order = RepairOrderGenerator.Generate(11, _catalog, sim.UnlockedEraCount);
        var result = RunFullLoop(sim, 11, 0);
        int expectedFee = Economy.FeeFor(order.eraId, order.faultIds.Length, 0, _catalog);
        Assert.That(result.pay, Is.EqualTo(expectedFee));
        Assert.That(result.rewardPct, Is.EqualTo(100));
        Assert.That(result.starsGained, Is.EqualTo(2));
        Assert.That(result.perfect, Is.True);
        // Wallet: parts charged on accept, fee paid on settle.
        Assert.That(wallet.stars, Is.EqualTo(2));
        Assert.That(wallet.money, Is.EqualTo(500 - CreditChargeFor(order, _catalog) + expectedFee));
        Assert.That(wallet.repairedPhones, Does.Contain(order.phoneId));
    }
    [Test]
    public void FullLoop_WithThreeMistakes_PaysPenalizedFee_AndGainsOneStar()
    {
        var wallet = new SimWallet { money = 500, stars = 0 };
        var sim = new RepairSim(_catalog, wallet);
        // Cap must mirror the sim's accept-time unlocked-era count (wallet stars = 0 -&gt; 1 era).
        var order = RepairOrderGenerator.Generate(23, _catalog, sim.UnlockedEraCount);
        var result = RunFullLoop(sim, 23, 3);
        Assert.That(result.perfect, Is.False);
        Assert.That(result.rewardPct, Is.EqualTo(90));
        Assert.That(result.starsGained, Is.EqualTo(1));
        // U153-D04 realignment: three mistakes also drain all three patience
        // pips, so the settle takes the non-fail x0.8 timeout on top of the
        // penalized fee (the order still settles - not a walkout).
        Assert.That(result.walkout, Is.False);
        Assert.That(result.pay,
            Is.EqualTo(Economy.FeeFor(order.eraId, order.faultIds.Length, 3, _catalog)
                * RepairSim.PatienceTimeoutPct / 100));
        Assert.That(wallet.stars, Is.EqualTo(1));
    }
    [Test]
    public void Parts_WithNoStock_AreChargedOnAccept()
    {
        var wallet = new SimWallet { money = 1000, stars = 0 };
        var sim = new RepairSim(_catalog, wallet);
```

```
    sim.AcceptNextCustomer(31);
    int expectedCharge = CreditChargeFor(sim.Session.order, _catalog);
    if (expectedCharge &gt; 0)
    {
        Assert.That(sim.Session.partCharged, Is.True, "no stock -&gt; purchase bookkeeping");
        Assert.That(sim.Session.partChargeCost, Is.EqualTo(expectedCharge));
    }
    else
    {
        Assert.That(sim.Session.partCharged, Is.False);
    }
    Assert.That(wallet.money, Is.EqualTo(1000 - expectedCharge));
}
[Test]
public void Parts_WithStock_ConsumeStockBeforeCharging()
{
    var order = RepairOrderGenerator.Generate(31, _catalog, 1);
    var firstPart = _catalog.Part(_catalog.Fault(order.faultIds[0]).partId);
    int firstCost = firstPart.costs[order.eraId];
    Assume.That(firstCost, Is.GreaterThan(0), "first part of seed 31 order must cost something");
    var wallet = new SimWallet { money = 1000, stars = 0 };
    wallet.partStock[firstPart.id] = 1;
    var sim = new RepairSim(_catalog, wallet);
    sim.AcceptNextCustomer(31);
    Assert.That(wallet.partStock[firstPart.id], Is.EqualTo(0), "stock consumed");
    int expectedCharge = ChargeFor(order, _catalog) - firstCost;
    Assert.That(sim.Session.partChargeCost, Is.EqualTo(expectedCharge));
    Assert.That(wallet.money, Is.EqualTo(1000 - expectedCharge));
}
[Test]
public void Determinism_SameSeedSequence_ProducesIdenticalOrdersAndWallets()
{
    var a = new RepairSim(_catalog, new SimWallet { money = 200, stars = 0 });
    var b = new RepairSim(_catalog, new SimWallet { money = 200, stars = 0 });
    for (int seed = 100; seed &lt; 120; seed++)
    {
        a.AcceptNextCustomer(seed);
        b.AcceptNextCustomer(seed);
        Assert.That(b.Session.order.eraId, Is.EqualTo(a.Session.order.eraId));
        Assert.That(b.Session.order.phoneId, Is.EqualTo(a.Session.order.phoneId));
        CollectionAssert.AreEqual(a.Session.order.faultIds, b.Session.order.faultIds);
        Assert.That(b.Session.partChargeCost, Is.EqualTo(a.Session.partChargeCost));
        WalkDeskToMenu(a, seed % 2);
        WalkDeskToMenu(b, seed % 2);
    }
    Assert.That(b.Wallet.money, Is.EqualTo(a.Wallet.money));
    Assert.That(b.Wallet.stars, Is.EqualTo(a.Wallet.stars));
    CollectionAssert.AreEqual(a.Wallet.repairedPhones, b.Wallet.repairedPhones);
}
[Test]
```

```
public void EraCap_OrdersNeverExceedUnlockedEras()
{
    var wallet = new SimWallet { money = 0, stars = 6 }; // 6 stars -&gt; 2 eras
    for (int seed = 1; seed &lt;= 50; seed++)
    {
        var sim = new RepairSim(_catalog, wallet);
        Assert.That(sim.UnlockedEraCount, Is.EqualTo(2));
        sim.AcceptNextCustomer(seed);
        Assert.That(sim.Session.order.eraId, Is.LessThan(2), "order must respect era cap");
    }
}
[Test]
public void Museum_TracksDistinctRepairedPhones()
{
    var sim = new RepairSim(_catalog, new SimWallet { money = 0, stars = 0 });
    var repaired = new List&lt;string&gt;();
    for (int seed = 200; seed &lt; 210; seed++)
    {
        sim.AcceptNextCustomer(seed);
        string phoneId = sim.Session.order.phoneId;
        WalkDeskToMenu(sim, 0);
        repaired.Add(phoneId);
    }
    CollectionAssert.AreEquivalent(repaired.Distinct().ToList(), sim.Wallet.repairedPhones);
    foreach (var id in sim.Wallet.repairedPhones)
    {
        Assert.That(_catalog.Phone(id), Is.Not.Null, "museum entry must resolve");
    }
}
[Test]
public void QuipKeys_PerfectDrawsHappy_MistakesDrawOk()
{
    var sim = new RepairSim(_catalog, new SimWallet { money = 0, stars = 0 });
    var perfect = RunFullLoop(sim, 301, 0);
    Assert.That(_catalog.Customers.happyQuipKeys, Does.Contain(perfect.quipKey));
    sim = new RepairSim(_catalog, new SimWallet { money = 0, stars = 0 });
    var sloppy = RunFullLoop(sim, 301, 1);
    Assert.That(_catalog.Customers.okQuipKeys, Does.Contain(sloppy.quipKey));
}
[Test]
public void StarsAccumulate_AndUnlockNextEraAtThreshold()
{
    var sim = new RepairSim(_catalog, new SimWallet { money = 0, stars = 0 });
    Assert.That(sim.UnlockedEraCount, Is.EqualTo(1));
    RunFullLoop(sim, 401, 0); // +2
    RunFullLoop(sim, 402, 0); // +2
    Assert.That(sim.UnlockedEraCount, Is.EqualTo(1), "4 stars still one era");
    RunFullLoop(sim, 403, 0); // +2 -&gt; 6 stars
    Assert.That(sim.Wallet.stars, Is.EqualTo(6));
    Assert.That(sim.UnlockedEraCount, Is.EqualTo(2), "6 stars unlock era 2");
```

```
        }
        [Test]
        public void WrongPhaseActions_Throw()
        {
            var sim = new RepairSim(_catalog);
            Assert.That(() =&gt; sim.ConfirmDiagnosis(), Throws.InvalidOperationException,
                "diagnosis from Menu must throw");
            Assert.That(() =&gt; sim.CompleteStep(true), Throws.InvalidOperationException,
                "step completion from Menu must throw");
            sim.AcceptNextCustomer(501);
            Assert.That(() =&gt; sim.AcceptNextCustomer(502), Throws.InvalidOperationException,
                "accept while a session is live must throw");
            Assert.That(() =&gt; sim.FinishRepair(), Throws.InvalidOperationException,
                "finish from Desk must throw");
        }
        [Test]
        public void SessionI18nKeys_AllResolve()
        {
            var sim = new RepairSim(_catalog);
            sim.AcceptNextCustomer(601);
            var order = sim.Session.order;
            Assert.That(G02Text.Has(order.customerNameKey), Is.True, "customer name key");
            Assert.That(G02Text.Has(order.openLineKey), Is.True, "open line key");
            foreach (var id in order.faultIds)
            {
                var fault = _catalog.Fault(id);
                Assert.That(G02Text.Has(fault.nameKey), Is.True, "fault name key");
                foreach (var s in fault.symptomKeys) Assert.That(G02Text.Has(s), Is.True);
                foreach (var step in fault.steps)
                {
                    Assert.That(G02Text.Has(_catalog.StepNameKey(step)), Is.True, "step name key");
                }
            }
        }
    }
}
// U153DepthD01D02Tests.cs
using System.Collections.Generic;
using NUnit.Framework;
using Sy.G02;
namespace Sy.G02.Tests
{
    /// &lt;summary&gt;
    /// U153 depth-repair sim tests (batch C399): D01 diagnosis-probe real decisions
    /// (three-path divergence, quick-boot rework, open-up teardown step, ask-details
    /// tip) + D02 inventory economy (credit markup 110%, prebuy discount 95%,
    /// deterministic stockout roll inside the [10,30]% DoD band).
    /// &lt;/summary&gt;
    public class U153DepthD01D02Tests
    {
```

```
GameCatalog _catalog;
[SetUp]
public void SetUp()
{
    _catalog = GameCatalog.Load();
    Assert.That(_catalog, Is.Not.Null, "catalog must load from Resources in batchmode");
    G02Text.Load();
}
static int[] RunProbe(GameCatalog catalog, int seed, int option)
{
    var sim = new RepairSim(catalog);
    sim.AcceptNextCustomer(seed);
    sim.ConfirmDiagnosis();
    sim.ResolveDiagnosis(option);
    sim.StartRepair();
    int guard = 0;
    while (!sim.Session.RepairDone &amp;&amp; guard &lt; 128) { sim.CompleteStep(true); guard++; }
    var result = sim.FinishRepair();
    return new[] { sim.Session.faults.Length, sim.Session.totalSteps, result.pay };
}
/// &lt;summary&gt;D01 DoD: three probe paths must diverge in &gt;= 2 measured dims
/// (revealed faults / bench steps / pay) on the same order.&lt;/summary&gt;
[Test]
public void D01_ThreeProbes_DivergeInAtLeastTwoDims()
{
    var rows = new List&lt;int[]&gt;();
    for (int opt = 0; opt &lt; 3; opt++) rows.Add(RunProbe(_catalog, 12345, opt));
    int dims = 0;
    for (int m = 0; m &lt; 3; m++)
    {
        bool differs = false;
        for (int r = 1; r &lt; rows.Count; r++) if (rows[r][m] != rows[0][m]) differs = true;
        if (differs) dims++;
    }
    Assert.That(dims, Is.GreaterThanOrEqualTo(2), "D01 DoD wants &gt;=2 diverging dims, got " + dims);
}
/// &lt;summary&gt;D01 DoD: over a seed batch the quick-boot path triggers rework
/// (miss rate &gt; 0) and every rework session settles at the 90% discount.&lt;/summary&gt;
[Test]
public void D01_QuickBoot_ReworkTriggersInBatch_AndDiscountsPay()
{
    int reworks = 0, twoFaults = 0;
    for (int seed = 1; seed &lt;= 60; seed++)
    {
        var sim = new RepairSim(_catalog);
        sim.AcceptNextCustomer(seed);
        if (sim.Session.faults.Length &lt; 2) continue;
        twoFaults++;
        sim.ConfirmDiagnosis();
        sim.ResolveDiagnosis(0);
```

```
            sim.StartRepair();
            int guard = 0;
            while (!sim.Session.RepairDone &amp;amp; guard &lt; 128) { sim.CompleteStep(true); guard++; }
            sim.FinishRepair();
            if (sim.Session.reworkTriggered)
            {
                reworks++;
                Assert.That(sim.Session.faults.Length,
                    Is.EqualTo(sim.Session.order.faultIds.Length),
                    "rework must surface every hidden fault");
            }
        }
        Assert.That(twoFaults, Is.GreaterThan(0), "batch must contain two-fault orders");
        Assert.That(reworks, Is.GreaterThan(0), "D01 DoD: quick-boot miss/rework rate must be &gt; 0");
    }
    /// &lt;summary&gt;D01: open up prepends exactly one teardown step (catalog untouched);
    /// ask details tips +20% and costs one patience pip.&lt;/summary&gt;
    [Test]
    public void D01_OpenUpAddsOneStep_AskDetailsTipsAndCostsPatience()
    {
        var sim = new RepairSim(_catalog);
        sim.AcceptNextCustomer(12345);
        int before = sim.Session.totalSteps;
        sim.ConfirmDiagnosis();
        sim.ResolveDiagnosis(1);
        Assert.That(sim.Session.totalSteps, Is.EqualTo(before + 1), "open up = +1 teardown step");
        var catalogFault = _catalog.Fault(sim.Session.faults[0].id);
        Assert.That(catalogFault.steps.Length, Is.EqualTo(before),
            "shared catalog fault steps must stay unmutated");
        var sim2 = new RepairSim(_catalog);
        sim2.AcceptNextCustomer(12345);
        sim2.ConfirmDiagnosis();
        sim2.ResolveDiagnosis(2);
        Assert.That(sim2.Session.tipPct, Is.EqualTo(120), "ask details tip propensity +20%");
        Assert.That(sim2.Session.patience, Is.EqualTo(2), "ask details costs one patience pip");
    }
    /// &lt;summary&gt;D02 DoD: out-of-stock accept charges the 110% credit markup;
    /// prebuy restocks at 95% so the spread between the two money paths &gt;= 5%.&lt;/summary&gt;
    [Test]
    public void D02_CreditMarkup110_PrebuyDiscount95_SpreadAtLeast5Pct()
    {
        var sim = new RepairSim(_catalog);
        sim.AcceptNextCustomer(12345);
        int era = sim.Session.order.eraId;
        int catalogCost = 0;
        foreach (var f in sim.Session.faults)
        {
            if (f == null || string.IsNullOrEmpty(f.partId)) continue;
            var part = _catalog.Part(f.partId);
            if (part != null &amp;amp; era &gt;= 0 &amp;amp; era &lt; part.costs.Length) catalogCost += part.costs
```

```
    }
    if (catalogCost &lt;= 0) Assert.Ignore("seed order carries no part cost");
    Assert.That(sim.Session.partCharged, Is.True, "empty stock must charge on accept");
    Assert.That(sim.Session.partChargeCost,
        Is.EqualTo(catalogCost * RepairSim.CreditMarkupPct / 100),
        "credit auto-charge = catalog cost + 110% markup");
    // Find a non-stockout seed for a part with era cost, then compare money paths.
    string probePart = null;
    foreach (var f in sim.Session.faults)
    {
        if (f != null &amp;&amp; !string.IsNullOrEmpty(f.partId)) { probePart = f.partId; break; }
    }
    int buySeed = -1;
    for (int s = 1; s &lt;= 100; s++)
    {
        var simProbe = new RepairSim(_catalog);
        if (!simProbe.IsPartSoldOut(s, probePart, era)) { buySeed = s; break; }
    }
    Assert.That(buySeed, Is.GreaterThan(0), "batch must contain non-stockout seeds");
    var wallet = new SimWallet();
    var simBuy = new RepairSim(_catalog, wallet);
    int moneyBefore = wallet.money;
    Assert.That(simBuy.BuyPartStock(probePart, era, buySeed), Is.True, "prebuy must succeed off stockout");
    int prebuyPrice = moneyBefore - wallet.money;
    Assert.That(prebuyPrice,
        Is.EqualTo(_catalog.Part(probePart).costs[era] * RepairSim.PrebuyDiscountPct / 100),
        "prebuy price = catalog cost * 95%");
    int creditPrice = _catalog.Part(probePart).costs[era] * RepairSim.CreditMarkupPct / 100;
    Assert.That((creditPrice - prebuyPrice) * 100 / creditPrice,
        Is.GreaterThanOrEqualTo(5), "D02 DoD: prebuy vs credit spread &gt;= 5%");
}
/// &lt;summary&gt;D02 DoD: stockout events block prebuy only, and the deterministic
/// roll lands inside the [10%,30%] trigger band over a seed grid.&lt;/summary&gt;
[Test]
public void D02_Stockout_BlocksPrebuyOnly_RateInside10To30Band()
{
    var sim = new RepairSim(_catalog);
    int soldOutSeeds = 0, soldOutSeed = -1;
    for (int s = 1; s &lt;= 200; s++)
    {
        if (!sim.IsPartSoldOut(s, "battery", 0)) continue;
        soldOutSeeds++;
        if (soldOutSeed &lt; 0) soldOutSeed = s;
    }
    Assert.That(soldOutSeed, Is.GreaterThan(0), "grid must contain stockout seeds");
    double rate = soldOutSeeds / 200.0;
    Assert.That(rate, Is.InRange(0.10, 0.30),
        "stockout trigger rate must stay inside the [10%,30%] DoD band, got " + rate.ToString("0.000"));
    var wallet = new SimWallet();
    var simBuy = new RepairSim(_catalog, wallet);
```

```
            int moneyBefore = wallet.money;
            Assert.That(simBuy.BuyPartStock("battery", 0, soldOutSeed), Is.False,
                "stockout must block prebuy");
            Assert.That(wallet.money, Is.EqualTo(moneyBefore), "blocked prebuy must not charge");
            Assert.That(wallet.partStock.TryGetValue("battery", out int n) &amp;&amp; n &gt; 0, Is.False,
                "blocked prebuy must not restock");
        }
    }
}
// U153DepthD03Tests.cs
using System;
using System.Collections.Generic;
using NUnit.Framework;
using Sy.G02;
namespace Sy.G02.Tests
{
    /// &lt;summary&gt;
    /// U153 depth-repair sim tests (batch C404): D03 desk tradeoff - daily arrival
    /// quota with day flip, over-quota swap star cost (real scarcity), decline order
    /// with slot burn, and all-accept vs picky two-strategy star/gold divergence.
    /// &lt;/summary&gt;
    public class U153DepthD03Tests
    {
        GameCatalog _catalog;
        [SetUp]
        public void SetUp()
        {
            _catalog = GameCatalog.Load();
            Assert.That(_catalog, Is.Not.Null, "catalog must load from Resources in batchmode");
            G02Text.Load();
        }
        bool IsRisky(int seed)
        {
            var probe = new RepairSim(_catalog);
            probe.AcceptNextCustomer(seed);
            return probe.Session.faults.Length &gt;= 2;
        }
        /// &lt;summary&gt;Perfect-repair desk strategy runner: picky=false accepts every
        /// order, picky=true declines risky (&gt;=2 faults) ones. Returns
        /// [money, stars, settles].&lt;/summary&gt;
        int[] RunDeskStrategy(List&lt;int&gt; seeds, bool picky)
        {
            var wallet = new SimWallet();
            var sim = new RepairSim(_catalog, wallet);
            int settles = 0;
            foreach (var seed in seeds)
            {
                sim.AcceptNextCustomer(seed);
                if (picky &amp;&amp; sim.Session.faults.Length &gt;= 2)
                {
```

```
            sim.DeclineOrder();
            continue;
        }
        sim.ConfirmDiagnosis();
        sim.StartRepair();
        int guard = 0;
        while (!sim.Session.RepairDone &amp;&amp; guard &lt; 128) { sim.CompleteStep(true); guard++; }
        sim.FinishRepair();
        settles++;
        sim.ReturnToMenu();
    }
    return new[] { wallet.money, wallet.stars, settles };
}
/// &lt;summary&gt;D03 DoD: same-seed swaps isolate the swap fee - first three
/// free, then each costs a star ([0,0,0,&gt;0,&gt;0] on the star channel);
/// same-seed money refund nets to zero on every swap.&lt;/summary&gt;
[Test]
public void D03_RerollOverQuota_CostsStarsAfterThreeFree()
{
    var wallet = new SimWallet();
    wallet.stars = 5;
    var sim = new RepairSim(_catalog, wallet);
    sim.AcceptNextCustomer(12345);
    var starCosts = new List&lt;int&gt;();
    var moneyCosts = new List&lt;int&gt;();
    int prevMoney = wallet.money;
    int prevStars = wallet.stars;
    for (int r = 0; r &lt; 5; r++)
    {
        sim.RerollCustomer(12345);
        moneyCosts.Add(prevMoney - wallet.money);
        starCosts.Add(prevStars - wallet.stars);
        prevMoney = wallet.money;
        prevStars = wallet.stars;
    }
    for (int r = 0; r &lt; 5; r++)
    {
        Assert.That(moneyCosts[r], Is.EqualTo(0),
            "same-seed swap refund must net money to zero on every swap");
    }
    for (int r = 0; r &lt; 3; r++)
    {
        Assert.That(starCosts[r], Is.EqualTo(0), "swaps 1-3 stay inside the free quota");
    }
    Assert.That(starCosts[3], Is.GreaterThan(0), "swap 4 exceeds the free quota and must cost a star");
    Assert.That(starCosts[4], Is.GreaterThan(0), "swap 5 exceeds the free quota and must cost a star");
}
/// &lt;summary&gt;D03: an over-quota swap with no spare stars is refused (real
/// scarcity) and the live session survives the refusal untouched.&lt;/summary&gt;
[Test]
```

```
public void D03_RerollOverQuota_NoSpareStars_ThrowsAndKeepsSession()
{
    var wallet = new SimWallet();
    var sim = new RepairSim(_catalog, wallet);
    sim.AcceptNextCustomer(7);
    sim.RerollCustomer(7);
    sim.RerollCustomer(7);
    sim.RerollCustomer(7);
    int moneyBefore = wallet.money;
    Assert.Throws&lt;InvalidOperationException&gt;(delegate { sim.RerollCustomer(7); },
        "over-quota swap without a spare star must be refused");
    Assert.That(sim.Phase, Is.EqualTo(SimPhase.Desk), "refused swap must leave the session intact");
    Assert.That(wallet.money, Is.EqualTo(moneyBefore), "refused swap must not refund or charge");
}
/// &lt;summary&gt;D03: decline refunds the accept charge in full (Desk -&gt; Menu)
/// and the arrival slot stays burned - no slot refund, no star cost.&lt;/summary&gt;
[Test]
public void D03_DeclineOrder_RefundsButBurnsArrivalSlot()
{
    var wallet = new SimWallet();
    var sim = new RepairSim(_catalog, wallet);
    int moneyBefore = wallet.money;
    sim.AcceptNextCustomer(12345);
    Assert.That(sim.RemainingArrivalQuota(), Is.EqualTo(RepairSim.DailyArrivalQuota - 1),
        "accept burns one arrival slot");
    sim.DeclineOrder();
    Assert.That(sim.Phase, Is.EqualTo(SimPhase.Menu), "decline returns to Menu");
    Assert.That(sim.Session, Is.Null, "decline closes the session");
    Assert.That(wallet.money, Is.EqualTo(moneyBefore), "decline refunds the accept charge in full");
    Assert.That(sim.RemainingArrivalQuota(), Is.EqualTo(RepairSim.DailyArrivalQuota - 1),
        "decline must NOT refund the burned arrival slot (real tradeoff)");
}
/// &lt;summary&gt;D03: the business day holds five arrival slots; the day flips
/// on the next accept and the free swap quota resets with it.&lt;/summary&gt;
[Test]
public void D03_DayFlipsAtFive_SwapQuotaResets()
{
    var wallet = new SimWallet();
    wallet.stars = 5;
    var sim = new RepairSim(_catalog, wallet);
    sim.AcceptNextCustomer(11);
    sim.RerollCustomer(11);
    sim.RerollCustomer(11);
    sim.RerollCustomer(11); // free quota burned on day one
    sim.DeclineOrder();
    for (int i = 1; i &lt; RepairSim.DailyArrivalQuota; i++)
    {
        sim.AcceptNextCustomer(20 + i);
        sim.DeclineOrder();
    }
```

```
            Assert.That(sim.RemainingArrivalQuota(), Is.EqualTo(0), "five slots burn the business day");
            int starsBefore = wallet.stars;
            sim.AcceptNextCustomer(30); // day flip
            Assert.That(sim.RemainingArrivalQuota(), Is.EqualTo(RepairSim.DailyArrivalQuota - 1),
                "day flip resets the arrival quota");
            sim.RerollCustomer(30);
            Assert.That(wallet.stars, Is.EqualTo(starsBefore),
                "first swap after the day flip is free again (quota resets with the flip)");
        }
        /// &lt;summary&gt;D03 DoD: bot "accept everything" vs "skip risky orders" must
        /// diverge in both gold and stars (the desk choice is not fake). Pre-D04
        /// baseline note: with no failure state built yet, all-accept simply banks
        /// the x1.5 double-fault fees - divergence is the assertion, not balance.&lt;/summary&gt;
        [Test]
        public void D03_AllAcceptVsPickyBot_DivergesInStarsAndGold()
        {
            // Deterministic series: first ten seeds plus two risky (&gt;=2 faults) ones
            // so the picky strategy provably skips paid work.
            var seeds = new List&lt;int&gt;();
            for (int s = 1; s &lt;= 10; s++) seeds.Add(s);
            int riskyFound = 0;
            for (int s = 11; s &lt;= 400 &amp;&amp; riskyFound &lt; 2; s++)
            {
                if (IsRisky(s)) { seeds.Add(s); riskyFound++; }
            }
            Assert.That(riskyFound, Is.EqualTo(2), "seed scan must find risky orders for the picky strategy");
            var all = RunDeskStrategy(seeds, false);
            var picky = RunDeskStrategy(seeds, true);
            Assert.That(all[0], Is.Not.EqualTo(picky[0]), "gold outcomes must diverge between the two strategies");
            Assert.That(all[1], Is.Not.EqualTo(picky[1]), "star outcomes must diverge between the two strategies");
            Assert.That(all[2], Is.GreaterThan(picky[2]), "picky skips the risky orders");
            Assert.That(all[0], Is.GreaterThan(picky[0]),
                "pre-D04 baseline: the x1.5 double-fault fees stay banked by all-accept");
        }
    }
}
// U153DepthD04Tests.cs
using System;
using System.Collections.Generic;
using NUnit.Framework;
using Sy.G02;
namespace Sy.G02.Tests
{
    /// &lt;summary&gt;
    /// U153 depth-repair sim tests (batch C406): D04 real-failure state - the
    /// angry zero-pay walkout at the per-era mistake tolerance, the patience
    /// timeout (non-fail x0.8, truly coupled with the D01 +20% tip = 96% net),
    /// the tolerance ladder {5,4,3,2}, and the rescue-retry near-miss seat
    /// (AdPoints g02_rescue_retry: tolerance +1, settle x0.8, once per order).
    /// &lt;/summary&gt;
```

```
public class U153DepthD04Tests
{
    GameCatalog _catalog;
    [SetUp]
    public void SetUp()
    {
        _catalog = GameCatalog.Load();
        Assert.That(_catalog, Is.Not.Null, "catalog must load from Resources in batchmode");
        G02Text.Load();
    }
    /// &lt;summary&gt;Fixture: start a session on the first seed (bounded scan)
    /// that rolls the target era with at least minSteps bench steps.
    /// stars = 30 unlocks all four eras for every probe.&lt;/summary&gt;
    RepairSim StartSessionAtEra(int eraId, int minSteps, out int seed)
    {
        for (int s = 1; s &lt;= 4000; s++)
        {
            var wallet = new SimWallet { stars = 30 };
            var sim = new RepairSim(_catalog, wallet);
            sim.AcceptNextCustomer(s);
            if (sim.Session.order.eraId == eraId &amp;&amp; sim.Session.totalSteps &gt;= minSteps)
            {
                seed = s;
                return sim;
            }
        }
        throw new AssertionException(
            "seed scan found no era " + eraId + " order with &gt;= " + minSteps + " steps");
    }
    /// &lt;summary&gt;D04 DoD: at the E3 tolerance (3) the third mistake on one
    /// order ends it with an angry zero-pay walkout - no museum entry, no
    /// reputation change, the day's arrival slot stays burned.&lt;/summary&gt;
    [Test]
    public void D04_WalkoutAtTolerance_ZeroPayNoMuseumSlotBurned()
    {
        int seed;
        var sim = StartSessionAtEra(2, 3, out seed);
        string phoneId = sim.Session.order.phoneId;
        int starsBefore = sim.Wallet.stars;
        int museumBefore = sim.Wallet.repairedPhones.Count;
        sim.ConfirmDiagnosis();
        sim.StartRepair();
        sim.CompleteStep(false);
        sim.CompleteStep(false);
        Assert.That(sim.RescueAvailable(), Is.True, "two mistakes = one from the E3 walkout");
        sim.CompleteStep(false); // third mistake = tolerance -&gt; angry walkout
        Assert.That(sim.Phase, Is.EqualTo(SimPhase.Settle), "walkout lands in Settle (Again loop reuse)");
        Assert.That(sim.LastResult.walkout, Is.True);
        Assert.That(sim.LastResult.pay, Is.EqualTo(0));
        Assert.That(sim.LastResult.starsGained, Is.EqualTo(0));
```

```
    Assert.That(sim.Wallet.stars, Is.EqualTo(starsBefore), "reputation stays unchanged on a walkout");
    Assert.That(sim.Wallet.repairedPhones.Count, Is.EqualTo(museumBefore), "no museum entry on a walkout");
    Assert.That(sim.Wallet.repairedPhones, Does.Not.Contain(phoneId));
    Assert.That(sim.RemainingArrivalQuota(), Is.EqualTo(RepairSim.DailyArrivalQuota - 1),
        "the arrival slot stays burned (D03 income cap holds)");
    sim.ReturnToMenu();
    Assert.That(sim.Phase, Is.EqualTo(SimPhase.Menu));
}
/// &lt;summary&gt;D04: patience timeout is a non-fail settle x0.8 and truly
/// couples with the D01 ask-details +20% tip - ask details (120%) plus a
/// drained patience bar (80%) nets 96% of the same-mistake baseline.&lt;/summary&gt;
[Test]
public void D04_PatienceTimeout_NonFail_TipNet96Pct()
{
    int seedA;
    var simA = StartSessionAtEra(0, 3, out seedA);
    simA.ConfirmDiagnosis();
    simA.ResolveDiagnosis(2); // ask details: tip 120, patience 3 -&gt; 2
    simA.StartRepair();
    simA.CompleteStep(false);
    simA.CompleteStep(false); // drains the rest: patience 0
    while (!simA.Session.RepairDone) simA.CompleteStep(true);
    var payA = simA.FinishRepair();
    int seedB;
    var simB = StartSessionAtEra(0, 3, out seedB);
    Assert.That(seedB, Is.EqualTo(seedA), "the bounded scan must be deterministic");
    simB.ConfirmDiagnosis();
    simB.ResolveDiagnosis(1); // open up: no tip, no patience cost
    simB.StartRepair();
    simB.CompleteStep(false);
    simB.CompleteStep(false);
    while (!simB.Session.RepairDone) simB.CompleteStep(true);
    var payB = simB.FinishRepair();
    Assert.That(simB.Session.patience, Is.EqualTo(1), "baseline keeps one pip - no timeout");
    Assert.That(simA.Session.patience, Is.EqualTo(0), "ask details + two mistakes drain the bar");
    Assert.That(payB.walkout, Is.False);
    Assert.That(payA.walkout, Is.False, "patience timeout is a non-fail settle");
    Assert.That(payA.pay, Is.GreaterThan(0));
    Assert.That(payA.pay,
        Is.EqualTo(payB.pay * RepairSim.AskDetailsTipPct / 100 * RepairSim.PatienceTimeoutPct / 100),
        "120% tip x 80% timeout = 96% net of the same-mistake baseline");
}
/// &lt;summary&gt;D04: the tolerance ladder is strictly decreasing E1 -&gt; E4,
/// anchored at "&gt;=3" = the E3 baseline value and E4 = 2 (measured).&lt;/summary&gt;
[Test]
public void D04_ToleranceLadder_StrictlyDecreasing()
{
    var sim = new RepairSim(_catalog, new SimWallet { stars = 30 });
    var tol = new List&lt;int&gt;();
    for (int e = 0; e &lt; _catalog.Eras.Length; e++) tol.Add(sim.MistakeTolerance(e));
```

```
    Assert.That(tol, Is.EqualTo(new[] { 5, 4, 3, 2 }), "ladder {5,4,3,2}");
    for (int e = 1; e &lt; tol.Count; e++)
    {
        Assert.That(tol[e], Is.LessThan(tol[e - 1]),
            "tolerance must strictly decrease from E" + e + " to E" + (e + 1));
    }
    Assert.That(sim.MistakeTolerance(2), Is.EqualTo(3), "'&gt;=3' DoD anchor = E3 baseline");
    Assert.That(sim.MistakeTolerance(3), Is.EqualTo(2), "E4 = 2 (measured)");
}
/// &lt;summary&gt;D04 near-miss seat: at one mistake from the E4 walkout the
/// rescue-retry extends this order's tolerance +1 with a settle x0.8,
/// is refused on a second call, and further mistakes still walk out.&lt;/summary&gt;
[Test]
public void D04_RescueRetry_SurvivesOncePerOrder_ThenStillWalks()
{
    int seed;
    var sim = StartSessionAtEra(3, 3, out seed);
    int faults = sim.Session.faults.Length;
    sim.ConfirmDiagnosis();
    sim.StartRepair();
    Assert.That(sim.RescueAvailable(), Is.False, "zero mistakes = no rescue seat yet");
    sim.CompleteStep(false); // mistake 1 = E4 tolerance - 1
    Assert.That(sim.RescueAvailable(), Is.True, "one mistake from the walkout opens the seat");
    sim.UseRescueRetry();
    Assert.That(sim.RescueAvailable(), Is.False, "the seat is consumed for this order");
    Assert.Throws&lt;InvalidOperationException&gt;(delegate { sim.UseRescueRetry(); },
        "rescue retry is once per order");
    sim.CompleteStep(false); // mistake 2 &lt; extended tolerance 3 -&gt; survives
    Assert.That(sim.Phase, Is.EqualTo(SimPhase.Repair), "the rescued order keeps bench time");
    while (!sim.Session.RepairDone) sim.CompleteStep(true);
    var result = sim.FinishRepair();
    Assert.That(result.walkout, Is.False);
    Assert.That(result.pay, Is.GreaterThan(0));
    Assert.That(result.pay,
        Is.EqualTo(Economy.FeeFor(3, faults, 2, _catalog) * RepairSim.RescueSettlePct / 100),
        "rescue settle = x0.8 of the two-mistake fee (patience 1 &gt; 0, no timeout stack)");
    // Same order again: after the rescue, mistakes still walk out (real failure kept).
    int seed2;
    var sim2 = StartSessionAtEra(3, 3, out seed2);
    Assert.That(seed2, Is.EqualTo(seed), "the bounded scan must be deterministic");
    sim2.ConfirmDiagnosis();
    sim2.StartRepair();
    sim2.CompleteStep(false);
    sim2.UseRescueRetry();
    sim2.CompleteStep(false); // mistake 2 &lt; 3: alive
    Assert.That(sim2.Phase, Is.EqualTo(SimPhase.Repair));
    sim2.CompleteStep(false); // mistake 3 = extended tolerance -&gt; walkout anyway
    Assert.That(sim2.Phase, Is.EqualTo(SimPhase.Settle));
    Assert.That(sim2.LastResult.walkout, Is.True, "further mistakes past the rescue still walk out");
    Assert.That(sim2.LastResult.pay, Is.EqualTo(0));
```

```
        }
        /// &lt;summary&gt;D04 existence testimony: a sloppy bot (all steps mistaken)
        /// meets the angry walkout across a deterministic mixed-era mini-run -
        /// the failure state is real, not theoretical.&lt;/summary&gt;
        [Test]
        public void D04_SloppyMiniRun_ProducesWalkouts()
        {
            int walkouts = 0;
            const int sessions = 12;
            for (int s = 1; s &lt;= sessions; s++)
            {
                var wallet = new SimWallet { stars = 30 };
                var sim = new RepairSim(_catalog, wallet);
                sim.AcceptNextCustomer(s * 101 + 7);
                sim.ConfirmDiagnosis();
                sim.StartRepair();
                int guard = 0;
                while (sim.Phase == SimPhase.Repair &amp;&amp; !sim.Session.RepairDone &amp;&amp; guard &lt; 128)
                {
                    sim.CompleteStep(false);
                    guard++;
                }
                if (sim.LastResult != null &amp;&amp; sim.LastResult.walkout) walkouts++;
                if (sim.Phase == SimPhase.Repair) sim.FinishRepair(); // short order survived
                sim.ReturnToMenu();
            }
            Assert.That(walkouts, Is.GreaterThanOrEqualTo(1),
                "the sloppy bot must meet the angry walkout on mixed-era orders (real failure state)");
        }
        /// &lt;summary&gt;D04: E4 is the harsh era - the tolerance is exactly two and
        /// the second mistaken step ends the order (near-miss at one).&lt;/summary&gt;
        [Test]
        public void D04_Era4Tolerance_TwoMistakesWalkout()
        {
            int seed;
            var sim = StartSessionAtEra(3, 2, out seed);
            Assert.That(sim.MistakeTolerance(3), Is.EqualTo(2), "E4 tolerance = 2 (measured)");
            sim.ConfirmDiagnosis();
            sim.StartRepair();
            sim.CompleteStep(false);
            Assert.That(sim.Phase, Is.EqualTo(SimPhase.Repair),
                "one mistake survives the harsh era - the near-miss seat");
            Assert.That(sim.RescueAvailable(), Is.True);
            sim.CompleteStep(false); // second mistake = E4 tolerance -&gt; walkout
            Assert.That(sim.Phase, Is.EqualTo(SimPhase.Settle));
            Assert.That(sim.LastResult.walkout, Is.True);
            Assert.That(sim.LastResult.pay, Is.EqualTo(0));
        }
    }
}
```